

The Google File System

Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung

Google*

摘要

我们设计并实现了 Google 文件系统 (GFS)，这是一个面向大规模分布式、数据密集型应用的可扩展分布式文件系统。它运行在廉价的通用硬件上，同时提供容错能力，并能为大量客户端提供很高的聚合性能。

虽然与之前的分布式文件系统有许多相同的目标，但是我们的设计主要受到两方面观察的驱动：一是我们应用的实际工作负载/访问模式，二是当时及预期中的技术环境。这与之前的一些文件系统假设有显著的不同。因此，我们重新审视了传统选择，并探索了截然不同的设计方向。

该文件系统已经成功满足了我们的存储需求。它已在 Google 内部被广泛部署，作为一种存储平台，用于生成和处理我们服务所需的数据，也用于需要大规模数据集的研究与开发工作。到目前为止，最大的集群在一千多台机器上的数千块磁盘中提供了数百 TB 的存储容量，并且有数百个客户端同时访问它。

本论文将介绍为支持分布式应用而扩展的文件系统接口，讨论设计中的多个方面，并报告微基准测试与真实使用场景下的测量结果。

1. 引言

我们设计并实现了 Google 文件系统 (GFS)，以满足 Google 数据处理需求快速增长的压力。GFS 与先前的分布式文件系统有许多共同目标，例如性能、可扩展性、可靠性和可用性。然而，它的设计由我们对当前及预期应用工作负载和技术环境的关键观察所驱动；这些观察与一些早期文件系统设计假设存在显著差异。我们重新审视了传统选择，并在设计空间中探索了截然不同的方向。

第一，组件故障是常态，而不是例外。该文件系统由数百乃至数千台采用廉价通用部件构建的存储机器组成，并由数量相当的客户端机器访问。组件的数量和质量几乎必然意味着：任意时刻总有部分组件无法工作，而且有些组件无法从当前故障中恢复。我们见过由应用程序缺陷、操作系统缺陷、人为错误，以及磁盘、内存、

连接器、网络和电源故障造成的问题。因此，持续监控、错误检测、容错和自动恢复必须成为系统不可或缺的一部分。

第二，按照传统标准来看，文件非常巨大。多 GB 级别的文件很常见。每个文件通常包含许多应用层对象，比如网页文档。当我们经常处理快速增长的、规模达到许多 TB 的数据集，并且这些数据集由数十亿个对象组成时，即使文件系统能够支持数十亿个约 KB 大小的小文件，管理这些小文件也会非常棘手。因此，像 I/O 操作大小、块大小这样的设计假设和参数，都必须重新审视。

第三，大多数文件通过追加新数据而不是覆盖已有数据来修改。文件内部的随机写入实际上几乎不存在。文件一旦写入，之后只会被读取，而且通常只会被顺序读取。多种数据都具有这些特征：有些数据可能形成大型数据仓库，供数据分析程序进行顺序扫描处理；有些可能是运行中应用持续生成的数据流；有些可能是归档数据；还有些可能是在一台机器上产生、由另一台机器同时或稍后处理的中间结果。鉴于超大文件的这种访问模式，追加成为性能优化和原子性保证的重点，而在客户端缓存数据块的吸引力则降低了（译者认为这里说的是传统客户端会尝试缓存数据，但是对于 GFS 这类通常顺序读取大量连续数据块的程序，缓存很不占优势：首先是文件太大，其次是缺少时间局部性，内容多半顺序读取一次后续就不读了，所以客户端完全不缓存数据块内容本身）。

第四，对应用程序和文件系统 API 进行协同设计，能够通过提高灵活性使整个系统受益。例如，我们放宽了 GFS 的一致性模型，在不向应用程序施加沉重负担的前提下，大幅简化了文件系统。我们还引入了原子追加操作，使多个客户端无需彼此额外同步，就能并发向同一文件追加数据。本文后面会更详细地讨论这些内容。

目前，多个 GFS 集群已针对不同用途部署。最大的集群拥有超过 1000 个存储节点、超过 300TB 的磁盘存储，并持续受到数百个不同机器上的客户端的高频访问。

2. 设计概览

2.1 设计假设

在为满足自身需求而设计文件系统时，我们遵循了一些既带来挑战、也带来机遇的假设。前文已提及一些关键观察；现在将更详细地说明这些假设。

- 系统由许多廉价且经常发生故障的通用组件构建而成。它必须持续监控自身，并能在日常运行中检测、容忍组件故障，并从中迅速恢复。
- 系统存储数量适中的大文件。我们预计会有数百万个文件，每个文件通常为 100MB 或更大。数 GB 大小的文件是常见情况，应被高效管理。系统必须支持小文件，但无需针对小文件进行优化。(译者注：因为这类存储往往是用来读取或写入大文件的，经典场景是大数据处理)
- 这些文件访问负载主要包括两类读操作：大型流式读取和小型随机读取。在大型流式读取中，单次操作通常会读取数百 KB，更常见的是读取 1MB 或更多。同一个客户端连续发起的操作，往往会沿着文件中的一段连续区域向前读取。小型随机读取通常会在任意 offset 处读取几 KB。注重性能的应用通常会把这些小读请求批量处理并排序，使读取位置在文件中稳定向前推进，而不是来回跳转。
- 这些文件访问负载还包含许多向文件追加数据的大型顺序写入。典型操作大小与读取操作相近。文件写入后很少再次被修改。系统支持在文件任意位置进行小写入，但不要求这种操作具有高性能。(译者认为，因为这类操作本身就不是 GFS 主打，GFS 主打大 block 顺序读写，而不是海量小文件读取，或者随机写，因此不必要被优化得很高效)
- 系统必须高效实现明确的语义，以支持多个客户端并发向同一文件追加数据。我们的文件常被用作生产者—消费者队列，或用于多路合并。数百个生产者每台机器运行一个，会并发向同一文件追加数据(译者注：经典的 *Fan-In* 模式，多个客户端向一个文件扇入数据)。以最小同步开销保证原子性至关重要。文件可能稍后才被读取，也可能正被某个消费者同时顺序读取。
- 高且持续的带宽比低延迟更重要。我们的大多数目标应用都重视以高速率批量处理数据；只有少数应用对单次读取或写入的响应时间有严格要求。

2.2 接口

GFS 提供了熟悉的文件系统接口，但并未实现 POSIX 这类标准 API。文件按目录进行层次化组织，并由路径名标识。系统支持常见的创建、删除、打开、关闭、读取和写入文件操作。

此外，GFS 还提供 **snapshot** 和 **record append** 操作。**snapshot** 能够以较低成本创建文件或目录树的 replica。**record append** 允许多个客户端并发向同一文件追加数据，同时保证每个客户端单次追加操作的原子性。**record append** 可以实现多路合并结果，以及允许许多客户端无需额外加锁便能同时追加数据的生产者—消费者队列。我们发现，这类文件对于构建大型分布式应用非常有价值。section 3.4 和 section 3.3 将分别进一步讨论 **snapshot** 和 **record append**。

2.3 架构

如 Figure 1 所示，一个 GFS 集群由单个 master 和多个 chunkserver 组成，并由多个客户端访问。它们通常都作为用户态进程的运行在普通的 Linux 服务器上。只要机器资源允许，并且能够接受由于运行可能不太稳定的应用程序代码而带来的可靠性下降，那么在同一台机器上同时运行 chunkserver 和 client 是很容易的。(译者注：事实上 Google 的 MapReduce 机器会和 chunkserver 混合部署，目的是尽量减少 Map 阶段的带宽消耗)

文件被划分为固定大小的 chunk。每个 chunk 都由 master 在创建时分配的、不可变且全局唯一的 64 位 chunk handle 标识(译者注：它不是文件 offset，也不是文件名，也不是磁盘地址，而是 master 给某个 chunk 分配的一个 64 位标识符)。chunkserver 将 chunk 作为 Linux 文件存储在本地磁盘上，并根据 chunk handle 和字节范围读取或写入 chunk 数据。为保证可靠性，每个 chunk 都会复制到多个 chunkserver 上。默认情况下，我们保存 3 个 replica；不过用户可以为文件命名空间的不同区域指定不同的复制级别。(译者认为此处指可以给不同的路径树前缀，设置单独的 replica 数，比如告诉/critical/这个路径下面的所有子文件都要 5 replica)

master 维护所有文件系统元数据，包括命名空间(译者认为 namespace 可以理解成目录树信息)、访问控制信息、从文件到 chunk 的映射，以及 chunk 的当前位置。它还控制全系统范围的活动，例如 chunk lease 管理、孤立 chunk 的垃圾回收，以及 chunk 在不同 chunkserver 之间的迁移。master 会定期通过 HeartBeat 消息与每个 chunkserver 通信，向其发送指令并收集其状态。

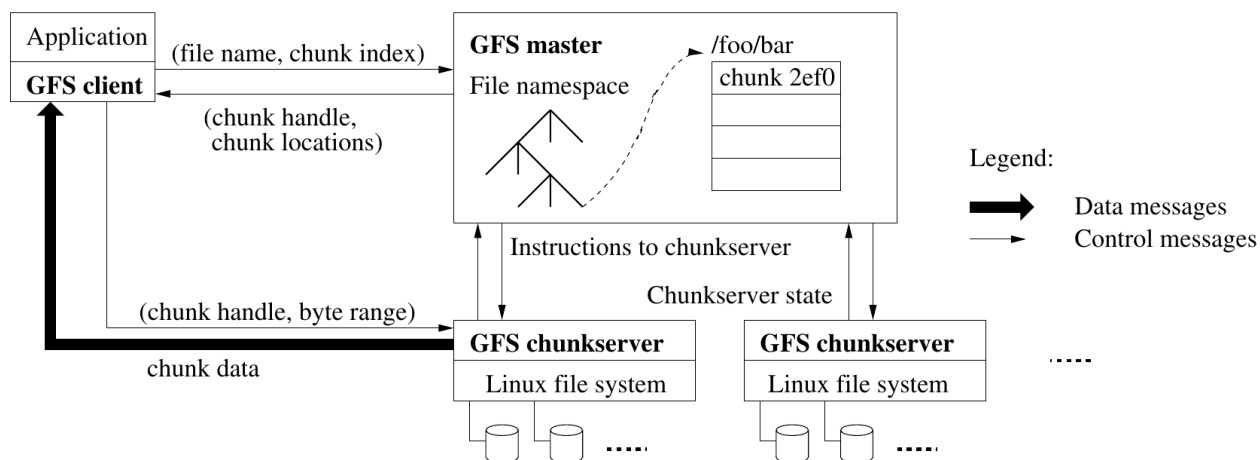


Figure 1. GFS 架构

每个应用程序中的 GFS 客户端代码实现了文件系统 API，并代表应用程序与 master 和 chunkserver 通信以读写数据。客户端在执行元数据操作时与 master 交互，但所有承载数据的通信都会直接发送到 chunkserver。我们不提供 POSIX API，因此无需 hook 到 Linux vnode 层。

客户端和 chunkserver 都不缓存文件数据。客户端缓存带来的收益很小，因为大多数应用会流式遍历超大文件，或其工作集大到无法被缓存。不缓存文件数据能够消除缓存一致性问题，从而简化客户端和整个系统（不过，客户端会缓存元数据）。chunkserver 也无需缓存文件数据，因为 chunk 以本地文件形式存储，Linux 的缓冲区缓存已经会将频繁访问的数据保留在内存中。

2.4 单个主控 (master) 服务器

单个 master 极大简化了我们的设计，并使 master 能够利用全局信息做出复杂的 chunk 放置和复制决策。然而，我们必须尽量减少它参与读取和写入的程度，以免它成为瓶颈。客户端绝不会经由 master 读取或写入文件数据。相反，客户端会询问 master 应联系哪些 chunkserver，并在有限时间内缓存这些信息，随后直接与 chunkserver 交互来完成许多操作。

下面结合 Figure 1 说明一次简单读取的交互过程。首先，客户端根据固定的 chunk 大小，将应用程序指定的文件名和字节 offset 转换为该文件中的 chunk 索引，译者注：chunk 索引计算公式：

$$\left\lfloor \frac{\text{offset}}{64 \times 1024 \times 1024} \right\rfloor$$

然后，它向 master 发送一个包含文件名和 chunk 索引的请求。master 返回相应的 chunk handle 及 replica 位置。客

户端使用文件名和 chunk 索引作为键，缓存这些信息。

接着，客户端向其中一个 replica 发送请求，通常会选择距离最近的 replica。请求中包含 chunk handle 和该 chunk 内的字节范围。在缓存信息过期或文件被重新打开之前，对同一个 chunk 的后续读取不再需要客户端与 master 交互。

实际上，客户端通常会在同一个请求中查询多个 chunk，而 master 也可以返回紧随这些 chunk 之后的 chunk 信息。这些额外信息几乎不增加成本，却能避免之后多次客户端与 master 之间的交互。

2.5 Chunk 大小

Chunk 大小是关键设计参数之一。我们选择了 64MB，这远大于典型的文件系统块大小。每个 chunk replica 都作为普通 Linux 文件存储在 chunkserver 上，并且只在需要时扩展。延迟 (lazy) 空间分配避免了内部碎片造成的空间浪费；而内部碎片可能正是人们反对采用如此大 chunk 大小的最大理由。（译者注：这里作者是在回应大 chunk 可能带来的空间浪费问题。64MB 是 chunk 的逻辑上限和管理粒度，并不意味着每创建一个 chunk replica 就会立刻占用 64MB 磁盘空间。由于 chunk replica 只是普通 Linux 文件，并且只会随着实际写入而扩展，因此即使最后一个 chunk 没有写满，也不会必然浪费完整的 64MB 空间。这里的内部碎片指的是固定大 chunk 可能导致的未使用空间浪费，而不是内存碎片）

较大的 chunk 大小具有几个重要优点。第一，同一 chunk 上的读写只需在开始时向 master 请求一次 chunk 位置信息，因此它减少了客户端与 master 交互的需求。对于我们的访问负载，这种减少尤其显著，因为应用大

多顺序读写大文件。即使进行小型随机读取，客户端也能轻松缓存一个多 TB 工作集中全部 chunk 的位置信息。第二，由于客户端更可能在一个大 chunk 上执行多次操作，它可以在较长时间内保持与 chunkserver 之间的持久 TCP 连接，从而降低网络开销（译者注：这里的意思是，应用通常会顺序访问大文件或其中一段连续区域。chunk 越大，客户端在同一个 chunk 内连续读写的机会越多，因此更可能复用同一个 chunkserver 连接，而不是频繁查询 master、切换 chunk 或重新建立 TCP 连接）。第三，它减少了存储在 master 上的元数据规模。这使我们能够将元数据保存在内存中，进而带来将在 section 2.6.1 讨论的其他优点。

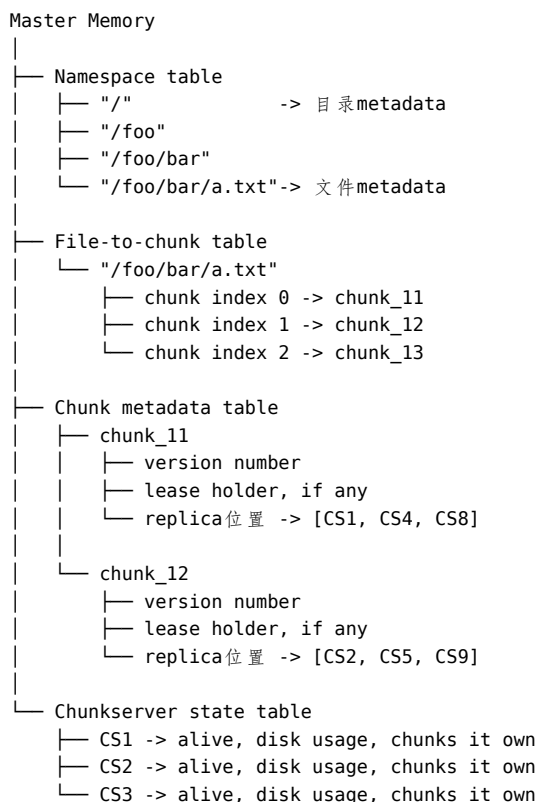
另一方面，即使采用延迟空间分配，大 chunk 大小也有缺点。一个小文件只包含少量 chunk，可能只有一个。如果许多客户端访问同一文件，存储这些 chunk 的 chunkserver 可能成为热点。实践中，热点并不是主要问题，因为我们的应用大多顺序读取包含多个 chunk 的大文件。

然而，在 GFS 最初被一个批处理队列系统使用时，确实出现了热点：一个可执行文件被作为单 chunk 文件写入 GFS，随后在数百台机器上同时启动。存储该可执行文件的少数 chunkserver 被数百个并发请求压垮。我们通过提高这类可执行文件的复制因子（译者注：简单理解为 replica 数量即可），并让批处理队列系统错开应用启动时间，解决了这个问题。一种潜在的长期方案是：在这类情况下允许客户端从其他客户端读取数据。

2.6 元数据

master 存储三类主要元数据：文件和 chunk 命名空间、从文件到 chunk 的映射，以及每个 chunk replica 的位置。所有元数据都保存在 master 的内存中。前两类元数据（命名空间和文件到 chunk 的映射）还会通过操作日志持久化。每次变更都会记录到存放在 master 本地磁盘、并复制到远程机器上的操作日志中。使用日志使我们能够简单、可靠地更新 master 状态，并避免在 master 崩溃时出现不一致的风险。master 不会持久化存储 chunk 位置信息。相反，master 会在启动时，以及每当某个 chunkserver 加入集群时，向各个 chunkserver 询问其持有的 chunk。

译者注：master 内存结构接近这样



2.6.1 内存数据结构

由于元数据存储在内存中，master 操作速度很快。此外，master 能够方便且高效地在后台定期扫描其全部状态。这种周期性扫描用于实现 chunk 垃圾回收、在 chunkserver 发生故障时重新复制 chunk，以及为平衡各 chunkserver 之间的负载和磁盘空间使用而迁移 chunk。section 4.3 和 section 4.4 会进一步讨论这些活动。

这种仅将元数据存储在内存中的方案有一个潜在顾虑：chunk 数量以及整个系统的容量，会受到 master 可用内存大小的限制。然而在实践中，这并不是严重限制。对于每个 64MB 的 chunk，master 维护的元数据不足 64 字节。大多数 chunk 都是满的，因为大多数文件包含许多 chunk，只有最后一个 chunk 可能未被完全填满。同样，文件命名空间数据通常每个文件也只需不足 64 字节，因为它通过前缀压缩紧凑地存储文件名。（译者注：一个 1TiB 文件，大约需要不到 1MiB 的 chunk 元数据；关于路径名压缩，可以考虑 trie 或者压缩前缀树压缩空间，现实生活中的 HDFS 没有压缩，并且 children 是二分查找，查找某个 children 的时候，复杂度 $O(\log n)$ ）

即使为了支持更大的文件系统，需要给 master 增加额外内存，这个成本也是值得的；因为将元数据保存在内存中，可以换来系统在简单性、可靠性、性能和灵活性上的收益。

2.6.2 chunk 位置

master 不会持久化记录哪些 chunkserver 持有某个给定 chunk 的 replica。它只会在启动时轮询 chunkserver 来获取这些信息。之后, master 能够保持信息最新, 因为它控制所有 chunk 放置操作, 并通过定期的 HeartBeat 消息监控 chunkserver 状态。

我们最初曾尝试在 master 上持久化保存 chunk 位置信息, 但后来认为, 在启动时以及之后定期向 chunkserver 请求这些数据要简单得多。这样就消除了 chunkserver 加入或离开集群、改名、发生故障、重启等情况下, 让 master 与 chunkserver 保持同步的问题。在拥有数百台服务器的集群中, 这些事件发生得非常频繁。

理解这一设计决策的另一种方式是: 对于自身磁盘上究竟持有哪些 chunk, chunkserver 拥有最终决定权。试图在 master 上维护这些信息的一致视图没有意义, 因为 chunkserver 上的错误可能导致 chunk 自行消失 (例如某块磁盘损坏并被禁用), 或者操作员可能会为 chunkserver 改名。

2.6.3 操作日志

操作日志包含了关键元数据变更的历史记录。它是 GFS 的核心组成部分。它不仅是元数据唯一的持久化记录, 同时也充当一条逻辑时间线, 用来定义并发操作的顺序。文件和 chunk, 以及它们的版本号 (见 section 4.5), 都由其创建时对应的逻辑时间唯一且永久地标识。

由于操作日志至关重要, 我们必须可靠地保存它, 并且只有在元数据变更被持久化之后, 才向客户端暴露这些变更。否则, 即使 chunk 本身仍然存在, 我们实际上也会丢失整个文件系统, 或丢失最近的客户端操作。因此, 我们将日志复制到多台远程机器上 (译者理解这里是为了容灾, 例如保证 master 机器磁盘坏了不会丢失持久化数据), 并且只有在相应日志记录已在本地和远程都 flush 到磁盘后, 才响应客户端操作。master 会在刷新前将多条日志记录批量合并, 从而减少 flush 和复制对系统整体吞吐量的影响。(译者注: 可以理解为多次 write fd 后才整体做一次 fsync, 因为一次 write 一次 sync 太慢。客户端不能只等到 master 执行了 write(fd, logRecord) 就收到成功响应。必须等到包含自己那条 log record 的 batch 已经在本地和远程都 flush 到 disk 才能返回 success response)

master 通过重放操作日志来恢复文件系统状态。为了尽量缩短启动时间, 我们必须让日志保持较小。当日志增长到一定大小后, master 会对自身状态创建 checkpoint。这样恢复时只需从本地磁盘加载最新 checkpoint, 并重放此后的少量日志记录。

checkpoint 采用紧凑的类 B 树形式 (译者注: 这里没有指针, 靠文件 offset 维护树结构), 可以直接映射到内存中 (译者注: 例如 mmap), 用于命名空间查找 (译者注: 因为这是存磁盘上的, 无法插入分裂, 所以只读), 无需额外解析。这会进一步加快恢复速度, 并提高可用性。

译者注:

论文提到了 checkpoint 是一种“读取友好型的”的可直接加载进内存的类 B 树文件。它如此设计有以下好处: 一是无需 parse, 恢复速度快; 二是 checkpoint 可以直接读, 载入内存直接可以 lookup 某个目录 entry; 三是一个节点能存很多数据, 树的层数低, 查询速度快, cache 好。

创建 checkpoint 可能需要一段时间, 因此 master 的内部状态经过专门设计, 可以在不延迟新到 mutation (译者注: mutation 可以理解为改变系统状态的操作。在本段 master checkpoint 的语境下, 它主要指改变 master 元数据状态的操作, 例如创建文件、删除文件、分配新 chunk、修改 file-to-chunk 映射、更新 chunk 版本号或 lease 状态等。普通的数据写入和追加主要由 chunkserver 执行; 只有当它们引发 master 元数据变化时, 才属于这里需要被 master operation log 记录并进入 checkpoint 的 mutation) 的情况下创建新 checkpoint。master 会切换到一个新的日志文件, 并由单独线程创建新 checkpoint。新 checkpoint 包含切换前的所有 mutation。对于拥有数百万文件的集群, 创建过程约需一分钟。完成后, checkpoint 会同时写入本地和远程磁盘。(译者注: 这里讲了构建 checkpoint 实际上依赖内存数据, 但是内存数据是线上使用的, 为了不 hang 住用户的请求, 这里有数据结构上的设计。译者认为可以采用“双 map 法” (类似 golang sync.Map 的 dirty), 构建开始的时候将当前 namespace 结构标记为只读称为 snapshot, 并且开启一个新的 namespace 结构 current, 构建线程可以读 snapshot 创建 checkpoint; 同时 master 主要业务线程可以读 snapshot, 读写 current 完成 mutation。checkpoint 彻底创建完毕后合并 current 和 snapshot; 还有一种可行的办法是开启一个背景进程, 专门用来读老 snapshot, 应用 op log, 定期制作新的 snapshot, 这种做法不侵入 master 主进程的业务逻辑, ChatGPT 告诉我这种方法类似 HDFS 的 CheckpointNode 旁路构建)

恢复时只需要最新的完整 checkpoint 及其后的日志文件。较旧的 checkpoint 和日志文件可以自由删除, 但我们会保留少量 replica 以防灾难性故障。checkpoint 创建期间发生故障不会影响正确性, 因为恢复代码能够检测并跳过不完整的 checkpoint。

2.7 一致性模型

GFS 采用了一种宽松的一致性模型。该模型能够很好地支持高度分布式的应用程序，同时仍能保持实现相对简单且高效。下面我们将讨论 GFS 提供的保证，以及这些保证对应用程序意味着什么。我们还会重点说明 GFS 如何维持这些保证，但将具体细节留给本文的其他部分。

2.7.1 GFS 提供的保证

文件命名空间 mutation(例如文件创建)是原子的。它们完全由 master 处理：命名空间锁保证原子性和正确性(section 4.1)；master 的操作日志定义了这些操作的全局全序(section 2.6.3)。

	Write	Record Append
Serial success	<i>defined</i>	<i>defined interspersed with inconsistent</i>
Concurrent successes	<i>consistent but undefined</i>	
Failure	<i>inconsistent</i>	

Table 1. 数据 mutation 后的文件区域状态

数据 mutation(译者注：包含 write 和 record append)之后，文件区域的状态取决于 mutation 类型、操作是成功还是失败，以及是否存在并发 mutation。Table 1 总结了这些结果。如果无论客户端从哪个 replica 读取，始终都能看到相同数据，则该文件区域是 consistent 的。一次文件数据 mutation 之后，如果该区域既 consistent，并且客户端能完整看到该 mutation 写入的内容，则该区域是 defined 的。当 mutation 成功且未受并发写入干扰时，受影响区域是 defined 的(因此也是 consistent)：所有客户端始终都会看到该 mutation 写入的内容。并发成功的 mutation 会使该区域 undefined 但保持 consistent：所有客户端看到相同数据，但这些数据可能并不反映任何一个 mutation 实际写入的内容。通常，它由多个 mutation 相互混杂的片段组成(译者注：consistent but undefined 是 concurrent successes 的结果，例如 client1 在 offset 处写入 AAAA，client2 在 offset 处写入 BBBB，有可能最终的结果是 AABB，最重要的原因是 mutation 可能刚好跨分片，从两个客户端的角度看来，最终 write 函数返回的都是成功，写出来的结果应当是 AAAA or BBBB，最后读出来却可能是 AABB 这样的混叠数据，这就是未定义的含义)。失败的 mutation 会使该区域 inconsistent(因此也 undefined)：不同客户端可能在不同时间看到不同的数据。下文将说明应用程序如何区分 defined 区域和 undefined 区域。应

用程序无需进一步区分 undefined 区域的不同种类。

数据 mutation 可以是 write 或 record append。写入会在应用程序指定的文件 offset 写入数据。即使存在并发 mutation，record append 也会至少一次(译者注：这里是伏笔，以后读者会知道如何处理重复数据保证幂等性)原子地追加数据(即“record”)，但追加 offset 由 GFS 选择(section 3.3)，相比之下，“常规”追加只是一次写入，写入位置是客户端认为的当前文件末尾 offset，GFS 会将实际 offset 返回给客户端；这个 offset 标记了一个 defined region 的起始位置，而该区域包含这条记录。此外，GFS 可能会在记录之间插入 padding 内容(译者注：原因是 record append 不能让一条 record 跨 chunk 边界；如果当前 chunk 剩余空间不够放下整条 record，GFS 就把当前 chunk 剩余部分填充掉，然后让客户端到下一个 chunk 重新 append。实现原子 append 的核心设计正是“一个 record 不能跨 chunk”，只有一个 record 在一台 chunk 上，并且依赖 chunk 的 pipeline 串行排队执行机制才能实现“原子 append”)，或者插入重复的记录。这些 padding 和重复的记录占据的区域被视为 inconsistent 区域，并且相对于用户数据量而言通常很小。

在一系列成功的 mutation 之后，经过 mutation 的文件区域保证是 defined 的，并且包含最后一次 mutation 写入的数据。GFS 通过两种方式实现这一保证：(a) 在所有 replica 上以相同顺序将 mutation 应用到某个 chunk(section 3.1)；(b) 使用 chunk 版本号检测因 chunkserver 宕机期间遗漏 mutation 而过期的 replica(section 4.5)。过期 replica 绝不会参与 mutation，也不会被 master 作为 chunk 位置返回给请求客户端。系统会在最早的机会对它们进行垃圾回收。

由于客户端会缓存 chunk 的位置信息，因此在这些缓存信息被刷新之前，客户端可能会从一个 stale replica 读取数据。这个时间窗口受到两个因素限制：一是缓存条目的超时时间，二是下一次打开该文件时会清除缓存中该文件的所有 chunk 信息。此外，由于我们的多数文件都是只追加的，所以 stale replica 通常返回的是一个过早的 chunk 结束位置(译者注：如果用户 append-only 而不 overwrite，当 stale replica 可能和 primary 断连后，客户端实际上最多从它这感知到 early EOF，而不是 outdated data，这不是特别“危险”)，而不是过期数据。当 reader 重试并联系 master 时，它会立即获得当前最新的 chunk 位置信息。

一次成功 mutation 过去很久后，组件故障当然仍可能损坏或销毁数据。GFS 通过 master 与所有 chunkserver

之间的定期握手识别失效的 `chunkserver`，并通过校验和检测数据损坏 (section 5.2)。一旦问题暴露，系统会尽快从有效 `replica` 恢复数据 (section 4.3)。只有在 GFS 来得及反应之前、通常也就是几分钟内，某个 `chunk` 的全部 `replica` 都丢失时，该 `chunk` 才会不可逆地丢失。即使在这种情况下，它也是不可用而非被损坏：应用程序会收到明确错误，而不是损坏的数据。

2.7.2 对应用程序的影响

GFS 应用可以通过一些简单技术适应宽松的一致性模型。这些技术原本就因其他目的而需要，包括依赖追加而非覆盖、创建 `checkpoint`，以及写入可自验证、可标识的 `record`。

几乎所有应用都会通过追加而非覆盖来修改文件。一种典型用法是：写入者从头到尾 (译者注：使用 `append`) 生成一个文件，并在写完全部数据后，原子地将文件重命名为永久名称；或者定期记录已成功写入的数据量，形成 `checkpoint`。`checkpoint` 还可以包含应用层校验和。(译者注：这里的 `checkpoint` 和之前提到的 `master checkpoint` 不是一件事，此处指的记录某一块数据的 `checksum` 和 `size`，告诉应用层这部分数据已经可以被读取使用，这种边处理边读取对很多应用很有帮助) 读取者只验证和处理截至最后一个 `checkpoint` 的文件区域，因为该区域已知处于 `defined` 状态。即使系统存在一致性和并发的问題 (译者注：这里指的是 `append+checkpoint` 的做法)，这套做法都很好地满足了我们的需求。与随机写入相比，追加的效率更高，也更能抵御应用程序故障。`checkpoint` 使写入者可以重启后断点续写，并避免读取者处理虽然已经成功写入、但从应用程序角度看仍不完整的文件数据。

另一种典型用法是：许多写入者并发向同一文件追加数据，以生成合并结果或充当生产者-消费者队列 (译者注：很多读者会误以为增加生产者就能提高单个文件的 `append` 速率，实际上不是的，在同一时刻某个 `chunk` 的 `mutation` 必须经过这个时刻“仅有一个”的 `primary` 排队，所以写入速率还是受制于 `primary` 的单机网络、磁盘瓶颈)。`record append` 的“至少追加一次”语义能够保留每个写入者的输出。读取者按如下方式处理偶发的 `padding` 和重复 `record`。写入者准备的每条 `record` 都包含校验和等额外信息，以便验证其有效性。读取者可以使用校验和识别并丢弃额外 `padding` 和 `record` 片段。如果读取者无法容忍偶发的重复 `record` (例如它们会触发非幂等操作)，则可以使用 `record` 中的唯一标识符将其过滤掉。这些唯一标识符通常本来也需要用于命名相应的应用实体，例如 Web 文档。这些 `record I/O` 相关功能 (除了重复

`record` 删除之外)，都被封装在我们的应用共享的库代码中，并且也适用于 Google 内部其他文件接口实现。借助这些机制，同一串 `record`，加上少量罕见的重复 `record`，总是会被交付给 `record reader`。

3. 系统交互

我们在设计系统时，力求将 `master` 参与所有操作的程度降至最低。在此背景下，下面将说明客户端、`master` 和 `chunkserver` 如何交互，以实现数据 `mutation`、原子 `record append` 和 `snapshot`。

3.1 `lease` 与 `mutation` 顺序

`mutation` 是一种会改变某个 `chunk` 的内容或元数据的操作，例如普通写入或追加操作。每个 `mutation` 都会在该 `chunk` 的所有副本上执行。我们使用 `lease` 来在各个副本之间维持一致的 `mutation` 顺序。`master` 会把某个 `chunk` 的 `lease grant` 给其中一个副本，我们称这个副本为 `primary`。`primary` 会为该 `chunk` 上的所有 `mutation` 选择一个串行顺序。所有副本在应用 `mutation` 时都遵循这个顺序。因此，全局 `mutation` 顺序首先由 `master` 选择的 `lease` 顺序决定；在同一个 `lease` 内，则由 `primary` 分配的序列号决定。

`lease` 机制旨在降低 `master` 的管理开销。`lease` 的初始超时时间为 60 秒。不过，只要 `chunk` 仍在发生 `mutation`，`primary` 就可以向 `master` 请求延期，并且通常能够无限期获得延期。这些延期请求和 `grant lease` 会捎带在 `master` 与所有 `chunkserver` 定期交换的 `HeartBeat` 消息中。`master` 有时会尝试在 `lease` 到期前撤销它 (例如，`master` 希望禁止对一个正在重命名的文件进行 `mutation` 时)。即使 `master` 失去了与某个 `primary` 的通信，它也可以在旧 `lease` 到期后，安全地向其他 `replica` `grant lease`。(译者注：这里的 `rename` 更准确地说是 `master` 对 `namespace` 映射的一次原子修改，即把某个文件对象从旧路径绑定到新路径，而不是复制或重建文件数据。由于 `GFS client` 可能缓存了 `chunk` 的 `primary/replica` 信息，并在已有 `lease` 有效期内绕过 `master` 继续向 `primary` 发起 `append/write`，`master` 如果希望 `rename` 成为一个清晰的发布/提交边界，就需要暂时禁止该文件上的 `mutation`。因此它可能在 `lease` 到期前主动 `revoke lease`，使基于旧缓存状态的写入失效，后续写入必须重新联系 `master`，并基于新的 `namespace` 状态重新获得授权。你看假如你已经 `rename` 成功了，但是去 `append` 原来的那个文件路径才是 `ok`，岂不奇怪？)

Figure 2 通过以下带编号的步骤，展示了一次写入操

作的控制流。

1. 客户端询问 master: 哪个 chunkserver 持有该 chunk 当前的 lease, 以及其他 replica 的位置。如果没有任何 replica 持有 lease, master 会向它选择的一个 replica grant lease(此步骤未在图中展示)。

2. master 返回 primary 的身份以及其他 secondary replica 的位置。客户端会缓存这些数据以供之后的 mutation 使用。只有当 primary 不可达, 或回复表示自己不再持有 lease 时, 客户端才需要再次联系 master。

3. 客户端将数据推送到所有 replica。推送顺序可以任意。每个 chunkserver 都会将数据保存在内部 LRU 缓冲区缓存中, 直到数据被使用或因老化而被逐出。通过将数据流与控制流解耦, 我们可以不受哪个 chunkserver 是 primary 的影响, 依据网络拓扑调度开销较大的数据流, 从而提升性能。section 3.2 会进一步讨论这一点。

4. 所有 replica 确认收到数据后, 客户端向 primary 发送写请求。该请求标识先前已推送到全部 replica 的数据。primary 为它收到的所有 mutation 分配连续序列号(译者注: master 元数据的 mutation 顺序号/operation log 顺序和每个 chunk 内 data mutation 的顺序是两套不同的顺序体系, 不能混为一个全局序列), 这些 mutation 可能来自多个客户端, 从而提供所需的串行化。它按照序列号顺序将 mutation 应用到自己的本地状态。

5. primary 将写请求转发给所有 secondary replica。每个 secondary replica 都按照 primary 分配的相同序列号顺序应用 mutation。

6. 所有 secondary 都会回复 primary, 表明自己已完成该操作。

7. primary 回复客户端。任一 replica 遇到的错误都会报告给客户端。发生错误时, 写入可能已经在 primary 和任意一个 secondary replica 子集上成功。(如果它在 primary 处失败, 就不会获得序列号, 也不会被转发。)客户端请求会被视为失败, 修改后的区域则处于不一致状态。我们的客户端代码会通过重试失败的 mutation 来处理这类错误。它会先对步骤 3 到步骤 7 进行若干次尝试; 若仍失败, 便从写入操作的开始处重新重试。

译者注: 可以考虑一种网络隔离场景来理解该机制。

假设 chunk 有 A、B、C 三个 replica, 当前版本号为 7, A 持有 lease 并作为 primary。此时 A 与 master、B、C 发生网络隔离, 但 client 仍能连接到 A。client 可能仍基于旧缓存认为 A 是 primary, 并向 A 发起 mutation。A 即使能在本地为该 mutation 排序并尝试执行, 也无法把该 mutation 转发给 B、C 并获得它们的确认, 因此该请求会

被报告为失败; 如果 mutation 只在部分 replica 上执行成功, 则被修改区域处于 inconsistent 状态, client 需要重试。

随着 A 与 master 失联, master 在旧 lease 过期后, 可以从仍然可达且 up-to-date 的 replica 中选择新的 primary, 例如 B, 并推进 chunk version number, 例如从 7 增加到 8。B、C 收到新版本后继续参与后续 mutation; A 因为错过了版本更新和后续 mutation, 恢复后会被 master 识别为 stale replica, 不会再被返回给 client, 也不会参与新的 mutation, 之后会被垃圾回收。

这里的关键不是多数派提交, 而是: 同一个 chunk 的 mutation 必须由当前有效 lease holder 排序, 并由所有参与的 replica 按相同顺序执行; 旧 primary 的 lease 到期后不能继续合法提交 mutation, 版本号则用于识别那些错过 mutation 的 stale replica。

这里还有一个特别关键的点, version 更准确地说是 chunk 的一个 epoch 标签。master 会用 version 判断哪些 replica 属于当前有效的副本。一次 mutation 需要由当前 lease holder 排序, 并由 master 当前认可的 up-to-date replicas 按相同顺序执行。如果某个 replica 在 version 推进期间不可用, 它恢复后因版本落后而被识别为 stale, 不再参与后续 mutation。简单理解, version 隐式蕴藏了一个“有效 replica”列表, 一次成功的 mutation 必须让这列表里的所有人都执行 mutation。

如果应用程序的写入很大, 或跨越了 chunk 边界, GFS 客户端代码会将其拆分为多个写入操作。它们都会遵循上述控制流, 但可能与其他客户端的并发操作交错, 并被其覆盖。因此, 这个共享的文件区域最终可能包含来自不同客户端的片段。不过, 各 replica 仍然相同, 因为每个单独操作都以相同顺序在所有 replica 上成功完成。正如 section 2.7 所述, 这会使文件区域处于 consistent 但 undefined 的状态。

3.2 数据流

我们将数据流与控制流解耦, 以高效利用网络。控制流从客户端流向 primary, 再流向所有 secondary; 而数据则以流水线方式, 沿着精心选择的 chunkserver 链线性推送。我们的目标是充分利用每台机器的网络带宽, 避免网络瓶颈和高延迟链路, 并尽量缩短推送全部数据所需的延迟。

为了充分利用每台机器的网络带宽, 数据会沿着一条 chunkserver 链线性推送(译者注: 例如 Client->A->B->C), 而不是以其他拓扑结构(例如树形结构)分发。这样,

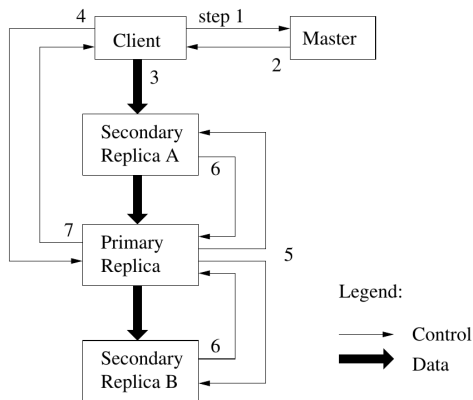


Figure 2. 写入控制流和数据流

每台机器的全部出站带宽都会用于尽快传输数据，而不是把出口带宽分摊给多个接收者（译者注：一台机器的出口带宽是有限的。实际上别的场景上，*Fan-Out* 在架构上一般不是什么好设计，译者在实际的工程中，例如 *IM* 群聊场景也有这样的体会，*Fan-Out* 架构会让群聊发送接口成为瓶颈——大量的 *CPU* 损耗和网络开销）。

为了尽可能避开网络瓶颈和高延迟链路（例如交换机之间的链路通常同时具备这两个问题），每台机器都会将数据转发给网络拓扑中距离最近、且尚未收到数据的机器。假设客户端正在向 chunkserver S1 到 S4 推送数据，它会先将数据发送给最近的 chunkserver，例如 S1。S1 会将数据转发给 S2 到 S4 中距离自己最近的 chunkserver，例如 S2。类似地，S2 会将数据转发给 S3 或 S4 中距离 S2 更近的一个，如此继续。我们的网络拓扑足够简单，因此可以根据 IP 地址准确估计这些“距离”。

最后，我们通过 TCP 连接上对数据传输进行流水线化来降低延迟。一旦 chunkserver 收到一部分数据，就会立即开始转发。流水线对我们特别有用，因为我们使用具有全双工链路的交换网络。立即发送数据不会降低接收速率。在没有网络拥塞时，向 R 个 replica 传输 B 字节数据的理想耗时为 $B/T + RL$ ，其中 T 是网络吞吐量， L 是字节在两台机器间传输的延迟。我们的网络链路通常为 100Mbps(T)，而 L 远小于 1ms。因此，理想情况下，分发 1MB 数据大约需要 80ms。

3.3 原子 record append

GFS 提供了一种称为 **record append** 的原子追加操作。在传统写入中，客户端指定数据要写入的 offset。对同一区域的并发写入无法串行化：该区域最终可能包含来自多个客户端的数据片段。而在 **record append** 中，客

户端只指定数据。GFS 会在它选择的 offset 处，至少一次原子地将数据追加到文件中（即作为一段连续字节序列），并将该 offset 返回给客户端。这类似于在 Unix 中向以 `O_APPEND` 模式打开的文件写入数据，即使是多个 writer 并发写入，也不会出现竞争而产生的“交叉文本”。（译者注：这里作者实际是在说 Unix `O_APPEND` 文件写入和 GFS 的 *record append* 语义相似，都是原子追加。然而，其实 GFS 的语义又和 Unix `O_APPEND` 不完全一样。第一点是 GFS 的 *record append* 保证的是至少一次，也就是两个线程并发追加内容，一个写 *hello*，一个写 *world*，那么真实在 GFS 里面的内容可能是 *hellohelloworld*，或者 *helloworldhelloworld*。你看，它虽然保证了一个 record 原子被插入，但是是多少次呢？答案是 GFS 只保证一次。而且最需要注意的是，插入后也许在 *chunk server* 不同的机器上读，会读出不同的结果。这里埋下伏笔，我将在后续提到这个 *badcase*。这个“乱序问题”很微妙，但是也可以被解决）

record append 被我们的分布式应用大量使用：不同机器上的许多客户端会并发向同一文件追加数据。如果使用传统写入，它们就需要额外进行复杂且昂贵的同步，例如通过分布式锁管理器完成同步。在我们的工作负载中，这类文件通常充当多生产者/单消费者队列，或保存来自许多不同客户端的合并结果。

译者注：

例如 *chunk10* 是文件的最后一个 *chunk*，*primary* 是 *server A*。客户端尝试向该 *chunk* 执行 *record append* 时，如果 *A* 发现该 *chunk* 的剩余空间不足以容纳完整 *record*，GFS 不会把这条 *record* 拆开跨 *chunk* 存放，而是由 *A* 将 *chunk10* 的剩余部分填充为 *padding*，并通知其他副本执行同样的 *padding mutation*。随后客户端需要在下一个 *chunk* 上重试 *record append*——也就是客户端需要请求 *master*，找 *master* 创建下一个 *chunk* 的元信息。

这里体现了 GFS *record append* 的一个设计取舍：为了保证一条 *record* 作为连续字节序列原子地落在某个 *chunk* 内，GFS 接受了 *chunk* 边界处可能产生 *padding* 和重试的代价。由于 GFS 写入流程是先 *push data* 到各副本，再请求 *primary* 执行 *mutation*，当大量客户端并发 *append* 且当前 *chunk* 接近满时，可能会出现一批已经 *push* 到旧 *chunk* 但最终无法写入的无效数据传输。

不过，这不意味着 GFS 不支持多生产者追加。相反，GFS 的 *record append* 正是为了支持多个客户端并发向同一文件追加记录。但单个文件的当前 *active chunk* 只有一个 *primary* 负责排序和分配 offset，因此单文件写入

吞吐不能随着 *writer* 数量线性扩展。若需要更高写入吞吐，通常应采用分片输出或多个队列文件，而不是让所有 *producer* 写入同一个文件。

record append 是一种 *mutation*，它遵循 section 3.1 中的控制流，只是在 *primary* 处增加了一些额外逻辑。客户端将数据推送到文件最后一个 *chunk* 的所有 *replica*，然后向 *primary* 发送请求。*primary* 会检查：在当前 *chunk* 执行 **record append** 是否会使其超过最大大小 (64MB)。如果会超过，*primary* 会将 *chunk* 填充到最大大小，通知 *secondary* 执行相同操作，并回复客户端，表示应在下一个 *chunk* 上重试该操作。(record append 被限制为至多最大 *chunk* 大小的四分之一，以将最坏情况下的碎片控制在可接受范围内)。如果 **record** 能够容纳在最大大小内 (这是通常情况)，*primary* 会将数据追加到自己的 *replica* 中，通知 *secondary* 在与 *primary* 完全相同的 *offset* 写入数据，最后向客户端回复成功。

如果 **record append** 在任一 *replica* 上失败，客户端会重试该操作。因此，同一 *chunk* 的 *replica* 可能包含不同数据，也可能全部或部分包含同一 **record** 的重复内容。*GFS* 并不保证所有 *replica* 在字节层面完全相同；它只保证数据至少会作为一个原子单元被写入一次。这一性质直接来自一个简单事实：要让操作报告成功，数据必须已在某个 *chunk* 的所有 *replica* 上的相同 *offset* 处被写入。并且在此之后，所有 *replica* 的长度至少会达到 **record** 结尾。因此，即使之后由不同 *replica* 成为 *primary*，任何未来 **record** 都会被分配到更高的 *offset*，或被分配到不同 *chunk*。按照我们的一致性保证，成功的 **record append** 操作写入数据的区域是已定义的 (因此也是一致的)，而其间的区域则是 *inconsistent* 的 (因此也是未定义的)。正如 section 2.7.2 所讨论的，我们的应用可以处理 *inconsistent* 区域。

译者注：回收之前的伏笔。读者可以去看 *Youtube* 观看 *Raft2020 Lec3 GFS* 视频，在 1h10 分处它讲解了一个由于失败重试，导致的“乱序问题”，它是一个很神奇的 *badcase*。

在真实的应用中，我们会使用 *append record* 来做追加的数据写入，读取则是从头读到已 *consistent* 的区域，然后从头到尾扫描“自标识”特征来识别 *record*——这样可以剔除无用 *padding* 或者 *record* 之间的 *garbage* 信息。但是这里教授的例子很微妙，它描述了一种非常奇妙的现象：多个 *writer append record* 同一个 *GFS File* 时，假如一次写入尝试因为“只有部分 *replica* 成功 *append*”而失败，后续客户端重试，会留下“脏数据”，而这个脏数

据也是合理的自标识数据。因此，就算业务层自己做了 *record* 自标识识别，也做了去重，但是在不同的机器可能会消费到不同的 *record* 顺序。所以怎么解决呢？一是可以干脆就不让多人写一个 *GFS* 文件了，事实上 *HDFS* 就是这么做的，它将 *lease* 做到了客户端和 *master* 上，保证同时只有一个客户端向一个文件 *append*；二是可以改造 *GFS*，我可以提供一些想法：可以尝试让所有机器都成为 *primary* 的复制状态机，每当 *primary* 上任的时候，强制让其它 *secondary* 同步自己经历过的 *mutation*，不允许出现分岔。读者可以多思考复制状态机以及 *Raft* 的 *No-op Entry* 优化，从这两个角度入手改造 *GFS*。

3.4 snapshot

snapshot 操作几乎可以瞬时创建一个文件或目录树 (即“source”) 的 *replica*，同时尽量减少对正在进行的 *mutation* 的干扰。用户使用它快速创建超大数据集的分支 *replica* (并且经常递归地创建这些 *replica* 的 *replica*)，或在试验某些之后可轻松提交或回滚的修改前，对当前状态创建检查点。

与 *AFS*[5] 类似，我们使用标准的 **copy-on-write** 技术实现 **snapshot**。当 *master* 收到 **snapshot** 请求时，首先会撤销即将创建 **snapshot** 的文件中所有 *chunk* 尚未到期的 *lease*。这样可以确保之后对这些 *chunk* 的任何写入，都必须与 *master* 交互以找到 *lease* 持有者。这使 *master* 有机会先创建该 *chunk* 的新 *replica*。

待 *lease* 被撤销或到期后，*master* 会将该操作记录到磁盘。然后，它通过复制 *source* 文件或 *source* 目录树的元数据，将这条日志记录应用到内存状态中。新创建的 **snapshot** 文件与 *source* 文件指向相同的 *chunk*。

在 **snapshot** 操作之后，客户端首次想要写入 *chunk C* 时，会向 *master* 请求当前 *lease* 持有者 (译者注：举个好例子，假如正在 *append*，客户端对 *primary* 有所缓存，但是在此刻 *chunk lease* 被收回，客户端写入时会发现 *not primary*，此时会去请求 *master* 寻找新的 *primary*)。*master* 发现 *chunk C* 的引用计数大于 1。它会暂缓回复客户端请求，转而选择一个新的 *chunk handle C'*。随后，它要求每个持有 *C* 当前 *replica* 的 *chunkserver* 创建一个名为 *C'* 的新 *chunk* (译者注：*lease* 是 *master* 调用 *chunkserver* 实现的，笔者认为创建 *C'* 时可以夹带 *copy-on-write* 信息，通知它进行复制)。通过在与原 *chunk* 相同的 *chunkserver* 上创建新 *chunk*，我们确保数据可以在本地复制，而无需经由网络传输 (我们的磁盘速度大约是 100Mb 以太网链路的 3 倍)。从此以后，请求处理与任何普通 *chunk* 都

没有区别：master 将新 chunk C 的 lease grant 给其中一个 replica，然后回复客户端。客户端随后可以像平常一样写入该 chunk，而不会察觉它刚刚是从一个现有 chunk 创建出来的。

译者注：snapshot 在用户视角那里其实就是像拷贝了一个文件夹，但是这个拷贝特别快，该文件夹下面的所有文件的所有 chunk lease 都会被 revoke 掉。读者可能会担心这里会有惊群效应客户端来拿 lease 会引发惊群效应。可以这样考虑：1. 读客户端不需要 lease，只有正在写的客户端需要来重新拿；2. 用户可以选择只 snapshot 一个小文件夹，尽量避免 snapshot 整个目录树，避免惊群。

4. master 服务器操作

master 执行所有命名空间操作。此外，它还管理整个系统中的 chunk replica：负责做出放置决策、创建新的 chunk 及其 replica，并协调各种全系统范围的活动，以保持 chunk 具有完整 replica、平衡所有 chunkserver 之间的负载，并回收未使用的存储空间。下面将分别讨论这些主题。

4.1 命名空间管理与锁

许多 master 操作可能耗时很长。例如，snapshot 操作必须撤销 snapshot 所覆盖的全部 chunk 上的 chunkserver lease。我们不希望这些操作运行时阻塞其他 master 操作。因此，我们允许多个操作同时处于活动状态，并使用命名空间区域上的锁来确保正确的串行化。

与许多传统文件系统不同，GFS 没有一种列出目录中全部文件的、按目录划分的数据结构（译者注：读者参考“元数据”一节的设计，它更像是维护的一个 *fullpath->metadata* 信息的映射，并没有按目录划分的树结构）。它也不支持同一个文件或目录的别名（即 Unix 术语中的硬链接或符号链接）。在逻辑上，GFS 将命名空间表示为一张从完整路径名映射到元数据的查找表。通过前缀压缩，这张表可以高效地存储在内存中。命名空间树中的每个节点（绝对文件名或绝对目录名）都关联着一个读写锁。

每个 master 操作在运行前都会获取一组锁。通常，若操作涉及 */d1/d2/.../dn/leaf*，它会对目录名 */d1*、*/d1/d2*、...、*/d1/d2/.../dn* 获取读锁，并对完整路径名 */d1/d2/.../dn/leaf* 获取读锁或写锁。请注意，*leaf* 可以是文件，也可以是目录，具体取决于操作类型。

下面说明这一加锁机制如何防止在将 */home/user* 创建 snapshot 到 */save/user* 期间，创建文件 */home/user/foo*。snapshot 操作会对 */home* 和 */save* 获取读锁，并

对 */home/user* 和 */save/user* 获取写锁。文件创建操作会对 */home* 和 */home/user* 获取读锁，并对 */home/user/foo* 获取写锁。由于两个操作都会尝试获取 */home/user* 上相互冲突的锁，它们会被正确地串行化。文件创建无需在父目录上获取写锁，因为不存在需要防止修改的“目录”或类似 inode 的数据结构。名称上的读锁已足以防止父目录被删除。

该加锁方案的一个优点是：它允许在同一目录中并发进行 mutation。例如，可以在同一目录中并发创建多个文件：每个操作都会获取目录名上的读锁，以及文件名上的写锁。目录名上的读锁足以防止该目录被删除、重命名或创建 snapshot。文件名上的写锁会将两次尝试创建同名文件的操作串行化。

由于命名空间可能拥有许多节点，读写锁对象会按需延迟分配，并在不再使用时删除。此外，为了防止死锁，锁会按一致的全序获取：先按命名空间树中的层级排序，同一层级内再按字典序排序。

4.2 replica 放置

一个 GFS 集群在多个层面上都具有高度分布性。它通常拥有分布在许多机器机架上的数百个 chunkserver。这些 chunkserver 又可能被来自同一机架或不同机架的数百个客户端访问。位于不同机架的两台机器之间通信时，可能需要跨越一个或多个网络交换机。此外，一个机架的进出带宽可能低于该机架内所有机器总带宽的总和。多层级的分布式结构给数据分布带来了独特挑战：既要保证可扩展性，又要保证可靠性和可用性。

chunk replica 放置策略有两个目的：最大化数据可靠性和可用性，以及最大化网络带宽利用率。对于这两个目标，仅将 replica 分散到不同机器上并不够。这样做只能防范磁盘或机器故障，并充分利用每台机器的网络带宽。我们还必须将 chunk replica 分散到不同机架。这确保即使整个机架受损或离线，某个 chunk 的部分 replica 仍能存活且可用。例如，机架可能因网络交换机或供电线路等共享资源故障而离线。这种做法还意味着，某个 chunk 的流量，尤其是读取流量，可以利用多个机架的聚合带宽（译者注：前面提到了机架的带宽是较小的，因此从多个机架下载会有更大的带宽，读者别忘了 *secondary* 机器同样可以下 chunk）。另一方面，写入流量必须跨越多个机架传输；这是我们愿意接受的权衡。

4.3 创建、重新复制与负载均衡

创建 chunk replica 有三个原因：创建 chunk、重新复制和重新均衡。

master 创建 chunk 时, 需要选择最初为空的 replica 应放置在哪里。它会考虑多个因素。(1) 我们希望将新 replica 放在磁盘空间利用率低于平均水平的 chunkserver 上。长期来看, 这会使各 chunkserver 之间的磁盘利用率趋于均衡。(2) 我们希望限制每个 chunkserver 上“最近”创建的数量。虽然创建本身开销很小, 但它能可靠地预示即将到来的大量写入流量: chunk 是在写入需要时创建的; 而在我们“追加一次、读取多次”的访问负载中, chunk 一旦被完全写入, 通常就几乎成为只读的。(3) 如前所述, 我们希望将一个 chunk 的 replica 分散到不同机架上。

一旦可用 replica 数量低于用户指定的目标值, master 就会重新复制该 chunk。出现这种情况可能有多种原因: 某个 chunkserver 变得不可用 (译者注: 比如“lease 与 mutation 顺序”提到的 primary 挂掉的场景, 若心跳检测超时, master 会认为它上面的 chunk 不再 available。后续如果需要一个新 lease 会重新选出一个 primary 并且赋予 lease, version 随即 +1, 此时其实该 chunk 就只有两个可用 replica 了。同理, 如果 secondary 挂掉, 此时 master 感知到了, 也会标记为 unavailable, 当需要 lease 的时候也会进行 grant new lease, 通知其它 chunk server: “目前只剩下这两个 replica 了”。读者可以清晰地察觉到这里其实有一个 lazy grant lease 的设计, 即使是 chunk 机挂了, 也不必马上重建 lease, 等需要的时候重建也很 ok); 它报告自己的 replica 可能已损坏; 其一块磁盘因错误被禁用; 或者复制目标值被提高。每个需要重新复制的 chunk 都会根据多个因素确定优先级。其中之一是它距离复制目标值的差距。例如, 丢失两个 replica 的 chunk 会比只丢失一个 replica 的 chunk 获得更高优先级。此外, 相比属于最近删除文件的 chunk, 我们优先重新复制仍然存活的文件的 chunk (section 4.4) (译者注: 读者可能会好奇为什么已经删除的文件也有复制优先级, 难道它也要复制? 是的, 因为 GFS 的删除其实是给一个 “Deleted At” 的标记然后改个名字, 到时间了再删除, 这样即使文件被错误地删除了也能抢救回来)。最后, 为了尽量减少故障对正在运行的应用程序的影响, 任何阻塞客户端进展的 chunk 都会被提高优先级。

master 选择优先级最高的 chunk, 并通过指示某个 chunkserver 直接从现有有效 replica 复制 chunk 数据, 来“克隆”它。新 replica 的放置目标与创建时类似: 均衡磁盘空间利用率、限制单个 chunkserver 上的同时进行的克隆操作, 并将 replica 分散到不同机架。为了防止克隆流量挤占客户端流量, master 会限制整个集群以及每个

chunkserver 上的同时进行的克隆操作数量。此外, 每个 chunkserver 都会通过限制向源 chunkserver 发出的读取请求数量, 来限制其为每次克隆操作使用的带宽。

最后, master 会定期重新对副本负载均衡: 它检查当前 replica 分布, 并迁移 replica 以获得更好的磁盘空间和负载均衡。通过这一过程, master 还能逐步填充新的 chunkserver, 而不是向新机器立即塞满大量新 chunk 以及随之而来的大量写入流量。新 replica 的放置条件与前文所述类似。此外, master 还必须选择要删除的现有 replica。通常, 它倾向于删除位于可用空间低于平均水平的 chunkserver 上的 replica, 以均衡磁盘空间使用率。译者注: 这里很好懂, 意思是一个新上任的 chunkserver 磁盘使用量很低按照优先级算法, replica 会成为 “hotspot”, 所以需要 master 进行渐进的、逐步的负载均衡。

4.4 垃圾回收

文件被删除后, GFS 不会立即回收其占用的物理存储空间。它只会在文件和 chunk 两个层级的定期垃圾回收过程中, 以延迟方式回收这些空间。我们发现, 这种做法使系统更加简单, 也更加可靠。

4.4.1 机制

当应用程序删除文件时, master 会像处理其他变更一样, 立即记录删除操作。但是, 它不会立即回收资源, 而是仅将文件重命名为包含删除时间戳的隐藏名称。在 master 定期扫描文件系统命名空间时, 如果这类隐藏文件已存在超过 3 天, 就会将其移除 (该时间间隔可配置)。在此之前, 文件仍可通过新的特殊名称读取, 也可以通过将其重命名回普通名称来恢复删除。当隐藏文件从命名空间中移除时, 其内存元数据也会被清除。这实际上切断了它与全部 chunk 之间的链接。

在对 chunk 命名空间进行类似的定期扫描时, master 会识别孤立 chunk (即无法从任何文件到达的 chunk), 并删除这些 chunk 的元数据。每个 chunkserver 会在与 master 定期交换的 HeartBeat 消息中, 报告自己持有的一部分 chunk。master 会回复所有已不再存在于自身元数据中的 chunk 标识。chunkserver 可以自由删除这类 chunk 的 replica。

4.4.2 讨论

在编程语言的语境中, 分布式垃圾回收是一个需要复杂解决方案的难题; 但在我们的场景中, 它相当简单。我们可以轻松识别全部指向 chunk 的引用: 它们位于完全由 master 维护的文件到 chunk 映射中。我们也能轻松

识别全部 chunk replica: 它们是每个 chunkserver 指定目录下的 Linux 文件。任何 master 未知的这类 replica 都是“垃圾”。

与急切删除相比, 采用垃圾回收方式回收存储空间有几个优点。第一, 在组件故障频繁的大规模分布式系统中, 它简单且可靠。chunk 创建可能只在部分 chunkserver 上成功, 从而留下 master 不知道的 replica。replica 删除消息可能丢失, 而 master 必须记得在自身或 chunkserver 发生故障后重新发送这些消息。垃圾回收提供了一种统一且可靠的方式, 用于清理所有已知无用的 replica。第二, 它将存储回收合并到 master 的常规后台活动中, 例如定期扫描命名空间以及与 chunkserver 握手。因此, 回收以批处理方式完成, 成本可以被分摊。此外, 只有在 master 相对空闲时才执行回收, 使 master 能够更及时地响应需要快速处理的客户端请求。第三, 延迟回收存储空间可以为意外且不可逆的删除提供一层安全保护。

根据我们的经验, 主要缺点是: 当存储空间紧张时, 这种延迟有时会妨碍用户精细调整存储使用。反复创建和删除临时文件的应用, 可能无法立即复用已释放的存储空间。我们通过以下方式处理这些问题: 如果一个已删除文件被显式再次删除, 就加速其存储回收。我们还允许用户对命名空间的不同部分应用不同的复制和回收策略。例如, 用户可以指定某个目录树内文件的全部 chunk 不进行复制, 并且任何被删除的文件都立即且不可逆地从文件系统状态中移除。

译者注:

GFS 的文件删除实际上就是打了一个“Deleted At”的标, 并且重命名, 等到三天后它才不可逆地被 master “遗忘”。在 master 遗忘后, 就通过心跳“懒加载”地告诉 chunk servers 某个文件已经删除了, chunk 此时才能被 chunk server 从磁盘中删除。这里说的是可以选择给不同目录设置不同的回收策略, 比如某个 critical 文件夹 5 天回收; 而 temp 文件夹立马回收。

此外这里还提到了之前就已经说过的可按目录配置副本数的功能, 可以给/tmp 设置较少的副本数(甚至是 1), 来让它少复制甚至不复制, 这样可以缓解磁盘资源紧张的问题。

4.5 过期 replica 检测

如果某个 chunkserver 发生故障, 并在宕机期间遗漏了对某个 chunk 的 mutation, 那么该 chunk 的 replica 可能变得过期 (stale)。对于每个 chunk, master 都会维护一

个 chunk 版本号, 用于区分最新 replica 和过期 replica。

每当 master 为某个 chunk grant 新的 lease 时, 都会提高该 chunk 的版本号, 并通知最新 replica。master 和这些 replica 都会将新版本号记录到各自的持久化状态中。这一过程发生在通知任何客户端之前, 因此也发生在客户端能够开始向该 chunk 写入之前。如果另一个 replica 当前不可用, 它的 chunk 版本号就不会被推进。当该 chunkserver 重启并报告其持有的 chunk 集合及对应版本号时, master 会发现它持有过期 replica。如果 master 看到的版本号高于自身记录的版本号, master 会假定自己在 grant lease 时曾发生故障, 并将较高版本视为最新版本。

译者注:

这里特别微妙, grant lease 过程中, master 先在内存中生成 $new_version = current_version + 1$, 随即 master 将该“换届”信息告知给 primary 和 secondary, 这些 replicas 都会持久化 version。接下来 master 就把这个 chunk version 的推进持久化写入 op log。请注意上述过程的任何部位都可能宕机, 因此要精细地考虑到所有边界情况。

在 master 重启后, 它做的事情是:

1. 从 checkpoint+op log 恢复 metadata, 包括 namespace 目录树, file->chunks mapping 以及 chunk handle 到 chunk 的 mapping 信息 (这里信息也包括 chunk version)。
2. 从所有 chunkserver 上报中汇聚信息, 包括 chunk version 和 locations, 补充自己的内存结构。若遇到比自己持久化的 version 小的, 直接视为 stale; 若遇到比自己 version 大的直接采用; 但是如果发现全部都比自己小的话, 只能告诉用户这个 chunk 不可用了。
3. 进入正轨, 处理请求, 补副本, 进行垃圾回收。

此外, 读者可以注意, 此处 master 假如发现机器宕机, 不论是 primary 还是 secondary, 都不会立刻进行新的 grant lease, 它只会默默地标记这个机器 unavailable 了。等下一次进行写入操作需要 lease 的时候, 才会进行新的 grant lease——这完完全全又是一个“懒加载”的思想, 避免一台机器挂掉需要处理大量的 grant lease, 而导致 master “惊群”。

master 会在定期垃圾回收中删除过期 replica。在此之前, 当它回复客户端的 chunk 信息请求时, 实际上会将过期 replica 视为完全不存在。作为另一层保护, master 会在通知客户端哪个 chunkserver 持有某个 chunk 的 lease 时, 附带该 chunk 的版本号; 或者, 在克隆操作中指示某个 chunkserver 从另一个 chunkserver 读取该 chunk 时, 也会附带版本号。客户端或 chunkserver 执行操作时会验

证版本号，从而确保其访问的始终是最新数据。

5. 容错与诊断

在设计该系统时，我们面临的最大挑战之一，是处理频繁发生的组件故障。组件的质量和数量共同决定了：这些问题是常态而非例外。我们既不能完全信任机器，也不能完全信任磁盘。组件故障可能导致系统不可用，或者更糟的是导致数据损坏。下面将讨论我们如何应对这些挑战，以及当问题不可避免地发生时，系统内置了哪些用于诊断问题的工具。

5.1 高可用性

在 GFS 集群的数百台服务器中，任意时刻都必然有部分服务器不可用。我们通过两种简单但有效的策略保持整个系统的高可用性：快速恢复和复制。

5.1.1 快恢复

无论是 master 还是 chunkserver，都被设计为无论以何种方式终止，都能在数秒内恢复状态并启动。事实上，我们不区分正常终止和异常终止；服务器通常只需直接杀死进程即可关闭。客户端和其他服务器只会经历一次轻微停顿：它们等待尚未完成的请求超时，重新连接到已重启的服务器，然后重试。section 6.2.2 报告了实际观察到的启动时间。

5.1.2 chunk 复制

如前所述，每个 chunk 都会复制到位于不同机架的多个 chunkserver 上。用户可以为文件命名空间的不同部分指定不同的复制级别，默认值为 3。当 chunkserver 离线，或通过校验和验证检测到 replica 损坏时（见 section 5.2），master 会按需克隆现有 replica，以确保每个 chunk 始终具有完整 replica。尽管复制一直很好地满足了我们的需求，但随着只读存储需求不断增长，我们正在探索其他跨服务器冗余形式，例如奇偶校验或纠删码。由于我们的流量主要由追加和读取构成，而不是小型随机写入，我们预计在这个非常松耦合的系统中实现这些更复杂的冗余方案虽有挑战，但仍然可控。

5.1.3 master 复制

master 状态通过复制来保证可靠性。它的操作日志和检查点会复制到多台机器上。只有当状态 mutation 对应的日志记录已在本地和所有 master replica 上刷新到磁盘后，该 mutation 才被视为已提交。为简单起见，一个 master 进程始终负责全部 mutation，以及垃圾回收等会在系统内部改变状态的后台活动。当它发生故障时，可

以几乎立即重启。如果它所在的机器或磁盘发生故障，GFS 外部的监控基础设施会在其他位置利用已复制的操作日志启动一个新的 master 进程。客户端只使用 master 的规范名称（例如 gfs-test）。该名称是一个 DNS 别名；如果 master 被迁移到另一台机器，该别名可以被修改。

此外，即使 primary master 宕机，“shadow” master 仍可向文件系统提供只读访问。它们是 shadow 而非 mirror，因为它们可能略微落后于 primary，通常只落后几分之一秒。对于未被积极 mutation 的文件，或不介意得到略旧结果的应用程序，shadow master 能够提高读取可用性。事实上，由于文件内容是从 chunkserver 读取的，应用程序不会观察到过期的文件内容。在短暂窗口中可能过期的是文件元数据，例如目录内容或访问控制信息。

为了保持信息更新，shadow master 会读取持续增长的操作日志 replica，并像 primary 一样，将完全相同的变更序列应用到自己的数据结构中。与 primary 相同，它会在启动时（以及此后不频繁地）轮询 chunkserver 以定位 chunk replica，并与它们频繁交换握手消息来监控状态。shadow master 只依赖 primary master 提供 replica 位置更新；这些更新源于 primary 关于创建和删除 replica 的决策。

译者注：这里描述地 shadow master 实际上“很弱”，它在 master 挂掉地时候只能提供读能力，并且会落后一些日志（论文里提到地几分之一秒），实际上读者可以去看看 VMWare FT 论文的思路，探讨是否能利用类似的思想设计出完全强一致的 backup。

5.2 数据完整性

每个 chunkserver 都使用校验和来检测已存储数据的损坏。考虑到一个 GFS 集群通常在数百台机器上拥有数千块磁盘，它会经常遇到磁盘故障；这些故障会在读取和写入路径上造成数据损坏或丢失。（其中一个原因见 section 7。）我们可以使用其他 chunk replica 从损坏中恢复，但通过比较不同 chunkserver 上的 replica 来检测损坏并不现实。此外，replica 之间存在差异也可能是合法的：GFS 的 mutation 语义，特别是前文讨论的原子 record append，并不保证 replica 完全相同。因此，每个 chunkserver 都必须通过维护校验和，独立验证自身 replica 的完整性。

一个 chunk 被划分为 64KB 的块，每个块都有对应的 32 位校验和。和其他元数据一样，校验和保存在内存中，并通过日志持久化存储；它们与用户数据分开保存。

对于读取操作，chunkserver 会在向请求方返回任何

数据前，验证与读取范围重叠的数据块的校验和。无论请求方是客户端还是另一个 *chunkserver*，都遵循这一规则。因此，*chunkserver* 不会将损坏传播到其他机器。如果某个块与记录的校验和不匹配，*chunkserver* 会向请求方返回错误，并向 *master* 报告该不匹配。随后，请求方会从其他 *replica* 读取，*master* 则会从另一个 *replica* 克隆该 *chunk*。新的有效 *replica* 就位后，*master* 会指示报告不匹配的 *chunkserver* 删除自己的 *replica*。

出于几个原因，校验和对读取性能影响很小。由于大多数读取至少跨越数个块，为了验证，我们只需额外读取并计算校验和的相对少量数据。*GFS* 客户端代码还会尽量将读取对齐到校验和块边界，以进一步降低这部分开销。此外，*chunkserver* 上的校验和查找与比较不需要任何 I/O，而校验和计算通常能够与 I/O 操作重叠执行。（译者注：这里其实说的是客户端可以先将范围对齐到 *block*，然后读取数据，读出来之后客户端可以缓存这块 *block*，后续其它小的读取就直接拿本地缓存的数据，不去找服务端读，可以减轻读 IO 和校验的开销。但是我感觉这个优化意义不大，而且之前 *GFS* 声称自己没有缓存 *data*，只缓存 *meta*，和之前的说法有点冲突了）

对于追加到 *chunk* 末尾的写入（而非覆盖已有数据的写入），校验和计算得到了高度优化，因为它们在我们的工作负载中占主导地位。我们只需增量更新最后一个未填满的校验和块的校验和，并为追加操作填满的任何全新校验和块计算新的校验和。即使最后一个未填满的校验和块已经损坏，而我们此时未能检测到，新校验和值也不会与已存储数据匹配；下次读取该块时，仍会像往常一样检测到损坏。

相反，如果写入覆盖了 *chunk* 中的一个现有范围，我们必须先读取并验证被覆盖范围的第一个和最后一个块，再执行写入，最后计算并记录新的校验和。如果在部分覆盖首尾块前不验证它们，新校验和可能会掩盖未被覆盖区域中原本存在的损坏。

在空闲期间，*chunkserver* 可以扫描并验证不活跃 *chunk* 的内容。这使我们能够检测很少被读取的 *chunk* 中的损坏。一旦检测到损坏，*master* 就可以创建一个新的未损坏 *replica*，并删除损坏 *replica*。这样可以避免一个不活跃但已损坏的 *chunk replica* 误导 *master*，使其以为某个 *chunk* 已经拥有足够数量的有效 *replica*。

译者注：

如何发现数据有错误？这里论文给出了两种方法：

1. *chunk server* 可以自己周期性地检查某个 *chunk* 的 *block checksum* 判断是否数据有损，如果有损，就可以

报告 *master*，这个 *replica* 有损。

2. 客户端主动读取的时候，检查 *block checksum*，如果有损则告知客户端切换到其它机器。之后可以向 *master* 上报 *replica* 有损。

5.3 诊断工具

广泛且细致的诊断日志，对问题隔离、调试和性能分析带来了极大帮助，而成本却很低。没有日志，很难理解机器之间短暂且无法复现的交互。*GFS* 服务器会生成诊断日志，记录许多重要事件（例如 *chunkserver* 上线和下线），以及全部 *RPC* 请求和回复。删除这些诊断日志不会影响系统正确性。不过，只要存储空间允许，我们会尽量保留这些日志。

RPC 日志包含网络上传输的精确请求与响应，但不包含正在读取或写入的文件数据。通过匹配请求和回复，并整理不同机器上的 *RPC* 记录，我们可以重建完整的交互历史，从而诊断问题。这些日志也可作为负载测试和性能分析的追踪记录。

日志记录对性能的影响很小（且远小于它带来的收益），因为这些日志以顺序、异步方式写入。最近发生的事件还会保存在内存中，可供持续在线监控。

译者注：

我的理解是客户端、*master* 以及 *chunkserver* 都可以记录日志，例如 *client* 可以创建 *trace-id*，用来串联一个请求的全链路，请求的 *Req* 和 *Resp* 可以写在本地；同理对于 *master* 和 *chunk server*，也会将 *RPC* 记录到它们本地；通过 *trace-id* 来串联一条调用，可以轻易地“全链路”追踪一个复杂的分布式过程。

6. 性能评测

本节将给出若干微基准测试，用以说明 *GFS* 架构和实现中固有的瓶颈；同时也会展示 Google 正在使用的真实集群的一些数据。

6.1 微基准测试

我们在一个由 1 个 *master*、2 个 *master replica*、16 个 *chunkserver* 和 16 个客户端组成的 *GFS* 集群上测量性能。请注意，这一配置是为便于测试而搭建的。典型集群拥有数百个 *chunkserver* 和数百个客户端。

所有机器均配置双 1.4GHz 的 PIII 处理器、2GB 内存、两块 80GB、5400 转/分的磁盘，以及一条连接到 HP2524 交换机的 100Mbps 全双工以太网链路。19 台 *GFS* 服务器全部连接到一台交换机，16 台客户端

机器全部连接到另一台交换机。这两台交换机之间通过一条 1Gbps 链路相连。

6.1.1 读

N 个客户端同时从文件系统读取数据。每个客户端从一个 320GB 的文件集合中随机读取一个 4MB 区域。该操作重复 256 次，因此每个客户端最终读取 1GB 数据。全部 chunkserver 的内存总量只有 32GB，因此我们预计 Linux 缓冲区缓存的命中率最多为 10%。我们的结果应当接近冷缓存条件下的结果。

Figure 3(a) 展示了 N 个客户端的聚合读取速率及其理论上限。该上限取决于先达到饱和的资源：当两台交换机之间的 1Gbps 链路饱和时，聚合速率上限为 125MB/s；当单个客户端的 100Mbps 网络接口饱和时，每个客户端的上限为 12.5MB/s。只有一个客户端读取时，观测到的读取速率为 10MB/s，即单客户端上限的 80%。当有 16 个读取者时，聚合读取速率达到 94MB/s，约为 125MB/s 链路上限的 75%，即每个客户端约 6MB/s。效率从 80% 下降到 75%，是因为随着读取者数量增加，多个读取者同时从同一个 chunkserver 读取数据的概率也会增加。

6.1.2 写

N 个客户端同时向 N 个不同文件写入数据。每个客户端通过一系列 1MB 写入，向一个新文件写入 1GB 数据。Figure 3(b) 展示了聚合写入速率及其理论上限。该上限稳定在 67MB/s，因为每个字节都需要写入 16 个 chunkserver 中的 3 个，而每个 chunkserver 的入站连接速率为 12.5MB/s。

单个客户端的写入速率为 6.3MB/s，约为该上限的一半。造成这一结果的主要原因是我们的网络栈。它与我们用于向 chunk replica 推送数据的流水线方案配合得不够好。从一个 replica 向另一个 replica 传播数据时产生的延迟，降低了整体写入速率。

当有 16 个客户端时，聚合写入速率达到 35MB/s(即每个客户端 2.2MB/s)，约为理论上限的一半。与读取情况相同，随着客户端数量增加，多个客户端并发写入同一个 chunkserver 的可能性也会提高。此外，16 个写入者发生冲突的可能性高于 16 个读取者，因为每次写入涉及 3 个不同 replica。

写入速度低于我们的预期。在实践中，这并未成为主要问题：尽管它会增加单个客户端所感知的延迟，但并不会显著影响系统向大量客户端提供的聚合写入带宽。

6.1.3 Record Append

Figure 3(c) 展示了 **record append** 的性能。N 个客户端同时向同一个文件追加数据。性能受存储该文件最后一个 chunk 的 chunkserver 网络带宽限制，与客户端数量无关。单个客户端时速率为 6.0MB/s；当客户端数增加到 16 个时，速率降至 4.8MB/s，主要原因是网络拥塞，以及不同客户端观察到的网络传输速率波动。

我们的应用通常会并发生成多个这样的文件。换言之，N 个客户端会同时向 M 个共享文件追加数据，其中 N 和 M 都达到数十或数百的规模。因此，实验中出现的 chunkserver 网络拥塞在实践中并不是显著问题，因为当某个文件对应的 chunkserver 忙碌时，客户端仍可以继续写入另一个文件。

6.2 真实世界的集群

我们现在考察 Google 内部正在使用的两个集群，它们可代表其他若干类似集群。集群 A 由一百多名工程师定期用于研究与开发。一个典型任务由人工用户发起，运行时间最长可达数小时。它读取数 MB 到数 TB 的数据，对数据进行转换或分析，并将结果写回集群。集群 B 主要用于生产数据处理。任务持续时间更长，并且只需偶尔人工干预，就能持续生成和处理数 TB 规模的数据集。在两种情况下，一个“任务”都由分布在多台机器上的许多进程构成，它们会同时读写大量文件。

6.2.1 存储

如 Table 2 的前五项所示，两个集群都拥有数百台 chunkserver，提供数十 TB 的磁盘空间，且磁盘使用率较高但尚未完全占满。“已用空间”包含所有 chunk replica。几乎所有文件都保存了 3 个 replica。因此，这两个集群分别存储了 18TB 和 52TB 的文件数据。

两个集群的文件数量相近，不过集群 B 中已删除文件所占比例更高，即已被删除或被新版本替换、但其存储空间尚未回收的文件。由于其中的文件往往更大，它也拥有更多 chunk。

Cluster	A	B
Chunkservers	342	227
Available disk space	72 TB	180 TB
Used disk space	55 TB	155 TB
Number of Files	735 k	737 k
Number of Dead files	22 k	232 k
Number of Chunks	992 k	1550 k
Metadata at chunkservers	13 GB	21 GB
Metadata at master	48 MB	60 MB

Table 2. 两个 GFS 集群的特征

6.2.2 元数据

所有 chunkserver 合计存储数十 GB 的元数据，其中大部分是用户数据 64KB 块的校验和。chunkserver 保存的另一类元数据只有 section 4.5 讨论过的 chunk 版本号。

master 保存的元数据要小得多，只有数十 MB，平均每个文件约 100 字节。这与我们的假设一致：在实践中，master 的内存大小不会限制系统容量。每个文件的大部分元数据是以前级压缩形式保存的文件名。其他元数据包括文件所有权和权限、文件到 chunk 的映射，以及每个 chunk 的当前版本。此外，对于每个 chunk，我们还保存当前 replica 位置和用于实现写时复制的引用计数。

每台服务器，无论是 chunkserver 还是 master，都只保存 50MB 到 100MB 的元数据。因此恢复速度很快：服务器只需从磁盘读取这些元数据数秒，便能够响应查询。不过，master 在从所有 chunkserver 获取 chunk 位置信息之前，仍会在一段时间内受到限制，通常为 30 到 60 秒。

6.2.3 读写速率

Table 3 展示了不同时间段内的读写速率。进行这些测量时，两个集群均已运行约一周。（这些集群不久前曾为升级到新版 GFS 而重启）

自重启以来，平均写入速率低于 30MB/s。我们进行这些测量时，集群 B 正处于一段写入活动突发期，产生约 100MB/s 的数据；由于写入会传播到 3 个 replica，这导致了 300MB/s 的网络负载。

Cluster	A	B
Read rate (last minute)	583 MB/s	380 MB/s
Read rate (last hour)	562 MB/s	384 MB/s
Read rate (since restart)	589 MB/s	49 MB/s
Write rate (last minute)	1 MB/s	101 MB/s
Write rate (last hour)	2 MB/s	117 MB/s
Write rate (since restart)	25 MB/s	13 MB/s
Master ops (last minute)	325 Ops/s	533 Ops/s
Master ops (last hour)	381 Ops/s	518 Ops/s
Master ops (since restart)	202 Ops/s	347 Ops/s

Table 3. 两个 GFS 集群的性能指标

读取速率远高于写入速率。正如我们所假设的，整体访问负载中读取操作多于写入操作。两个集群当时都处于高强度读取活动期间。特别是，集群 A 在此前一周一直维持 580MB/s 的读取速率。它的网络配置可支持 750MB/s，因此资源利用较为高效。集群 B 可支持 1300MB/s 的峰值读取速率，但其应用实际只使用了 380MB/s。

6.2.4 master 负载

Table 3 还表明，发送给 master 的操作速率约为每秒 200 到 500 次。master 可以轻松处理这一速率，因此不会

成为这些访问负载的瓶颈。

在较早版本的 GFS 中，master 偶尔会成为某些访问负载的瓶颈。它大部分时间都在顺序扫描大型目录（其中包含数十万个文件），以查找特定文件。此后，我们修改了 master 的数据结构，使其能够在命名空间中进行高效的二分查找。现在，它可以轻松支持每秒数千次文件访问。如有必要，我们还可以在命名空间数据结构之前设置名称查找缓存，进一步提高速度。

6.2.5 恢复时间

在一个 chunkserver 发生故障后，部分 chunk 会变为 replica 不足，必须通过克隆恢复其 replica 数量。恢复所有这类 chunk 所需的时间取决于可用资源数量。在一项实验中，我们终止了集群 B 中的一台 chunkserver。该 chunkserver 保存约 15,000 个 chunk，共含 600GB 数据。为限制对运行中应用的影响，并为调度决策预留余量，我们的默认参数将该集群限制为最多同时执行 91 个克隆操作（chunkserver 数量的 40%），且每个克隆操作最多可使用 6.25MB/s(50Mbps) 的带宽。所有 chunk 在 23.2 分钟内恢复完成，有效复制速率为 440MB/s。

在另一项实验中，我们终止了两台 chunkserver，每台大约保存 16,000 个 chunk 和 660GB 数据。这次双重故障使 266 个 chunk 只剩下一个 replica。我们以更高优先级克隆这 266 个 chunk，并在 2 分钟内全部将其恢复到至少 2 倍 replica，使集群能够在不丢失数据的情况下承受另一台 chunkserver 故障。

6.3 访问负载拆解

本节将详细拆解两个 GFS 集群上的访问负载。这两个集群与 section 6.2 中的集群具有可比性，但并不完全相同。集群 X 用于研究与开发，集群 Y 用于生产数据处理。

6.3.1 方法与注意事项

这些结果只包含由客户端发起的请求，以便反映我们的应用为整个文件系统产生的访问负载。它们不包括为执行客户端请求而产生的服务器间请求，也不包括内部后台活动，例如转发写入或负载再平衡。

I/O 操作的统计数据基于从 GFS 服务器记录的实际 RPC 请求中，以启发式方法重建的信息。例如，GFS 客户端代码可能会将一次读取拆分为多个 RPC，以提高并行度；我们据此推断出原始读取。由于我们的访问模式高度程式化，我们预计任何误差都会落入噪声范围。由应用显式记录日志或许能提供稍微更准确的数据，但从实际操作角度看，重新编译并重启数千个正在运行的客

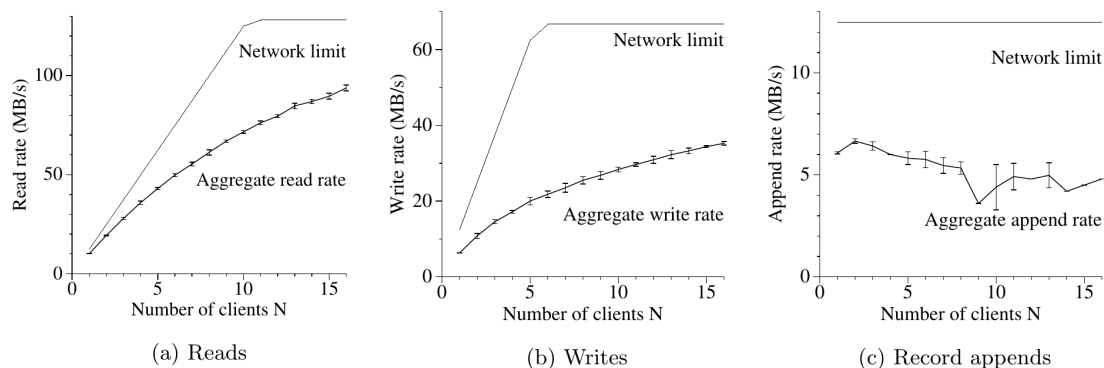


Figure 3. 聚合吞吐量。上方曲线表示网络拓扑带来的理论上限；下方曲线表示测得的吞吐量。误差线表示 95% 置信区间；由于测量方差较低，部分误差线难以辨认。

户端并不可行，而且从如此多机器收集结果也十分繁琐。

不应过度泛化我们的访问负载。由于 Google 同时完全控制 GFS 及其应用，应用往往会针对 GFS 进行调优；反过来，GFS 也是为这些应用设计的。这种相互影响也可能存在于通用应用之间。

Operation Cluster	Read		Write		Record Append	
	X	Y	X	Y	X	Y
0K	0.4	2.6	0	0	0	0
1B..1K	0.1	4.1	6.6	4.9	0.2	9.2
1K..8K	65.2	38.5	0.4	1.0	18.9	15.2
8K..64K	29.9	45.1	17.8	43.0	78.0	2.8
64K..128K	0.1	0.7	2.3	1.9	< .1	4.3
128K..256K	0.2	0.3	31.6	0.4	< .1	10.6
256K..512K	0.1	0.1	4.2	7.7	< .1	31.2
512K..1M	3.9	6.9	35.5	28.7	2.2	25.5
1M..inf	0.1	1.8	1.5	12.3	0.7	2.2

Table 4. 按操作大小划分的操作占比 (%)

6.3.2 chunkserver 访问负载

Table4 展示了按大小划分的操作分布。读取大小呈现双峰分布。小读取 (小于 64KB) 来自寻道密集型客户端，它们会在巨大文件中查找小片数据。大读取 (大于 512KB) 则来自对整个文件进行的长时间顺序读取。

集群 Y 中有相当数量的读取完全不返回数据。我们的应用，特别是生产系统中的应用，常将文件用作生产者-消费者队列。生产者并发向文件追加数据，而消费者读取文件末尾。当消费者的速度超过生产者时，偶尔会没有数据返回。集群 X 较少出现这种情况，因为它通常用于短生命周期的数据分析任务，而非长生命周期的分布式应用。

写入大小同样呈现双峰分布。大写入 (大于 256KB) 通常源于写入端进行了较大规模的缓冲。缓冲数据较少、更频繁地进行 checkpoint 或同步、或者生成数据较少的写入端，会产生较小的写入 (小于 64KB)。

对于 **record append**，集群 Y 中大型 **record append** 所占比例远高于集群 X，因为使用集群 Y 的生产系统针对 GFS 进行了更积极的调优。

Table5 展示了不同大小操作传输的数据总量。对于所有类型的操作，较大的操作 (大于 256KB) 通常占据大部分传输字节。由于随机寻道访问负载，小读取 (小于 64KB) 也会传输一部分虽小但不可忽略的读取数据。

Operation Cluster	Read		Write		Record Append	
	X	Y	X	Y	X	Y
1B..1K	< .1	< .1	< .1	< .1	< .1	< .1
1K..8K	13.8	3.9	< .1	< .1	< .1	0.1
8K..64K	11.4	9.3	2.4	5.9	2.3	0.3
64K..128K	0.3	0.7	0.3	0.3	22.7	1.2
128K..256K	0.8	0.6	16.5	0.2	< .1	5.8
256K..512K	1.4	0.3	3.4	7.7	< .1	38.4
512K..1M	65.9	55.1	74.1	58.0	.1	46.8
1M..inf	6.4	30.1	3.3	28.0	53.9	7.4

Table 5. 按操作大小划分的传输字节数占比 (%)。对于读取操作，大小指实际读取并传输的数据量，而非请求的数据量。如果读取操作尝试读取超过文件末尾的数据，这两者可能不同；按照设计，这种情况在我们的访问负载中并不少见。

Cluster	X	Y
Open	26.1	16.3
Delete	0.7	1.5
FindLocation	64.3	65.8
FindLeaseHolder	7.8	13.4
FindMatchingFiles	0.6	2.2
All other combined	0.5	0.8

Table 6. 按类型划分的 master 请求占比 (%)

6.3.3 append vs write

record append 被广泛使用，尤其是在我们的生产系统中。对于集群 X，按传输字节数计算，写入与 **record append** 的比例为 10:1；按操作次数计算，比例为 8:1。

对于用于生产系统的集群 Y，这两个比例分别为 3.7:1 和 2.5:1。此外，这些比例表明，两个集群中的 **record append** 通常都比 **write** 更大。然而，对于集群 X，在测量期间 **record append** 的总体使用量相当低，因此结果很可能被一两个采用特定缓冲区大小选择的应用所扭曲。

正如预期，我们的数据 **mutation** 访问负载以追加而非覆盖为主。我们测量了 **primary replica** 上被覆盖的数据量。这近似于客户端有意覆盖此前写入的数据，而非追加新数据的情形。对于集群 X，覆盖操作占 **mutation** 字节数的不足 0.0001%，占 **mutation** 操作数的不足 0.0003%。对于集群 Y，这两个比例均为 0.05%。尽管这一比例极小，但仍高于我们的预期。结果发现，大部分覆盖来自客户端因错误或超时而进行的重试。它们本身并不属于访问负载，而是重试机制带来的结果。

6.3.4 master 访问负载

Table 6 展示了发送给 master 的请求按类型划分的构成。大多数请求用于读取时获取 **chunk** 位置 (**FindLocation**)，以及进行数据 **mutation** 时获取 **lease** 持有者信息 (**FindLeaseLocker**)。

集群 X 与 Y 的 **Delete** 请求数量显著不同，这是因为集群 Y 存储的生产数据集会定期重新生成并由新版本替换。其中一部分差异还隐藏在 **Open** 请求数量的差异中，因为文件的旧版本可能会在从头以写入模式打开时被隐式删除 (按 Unix 的 **open** 术语，即“w”模式)。

FindMatchingFiles 是一种支持 **ls** 及类似文件系统操作的模式匹配请求。与其他发往 master 的请求不同，它可能需要处理命名空间中的很大一部分，因此开销可能较高。集群 Y 更频繁地出现此类请求，因为自动化数据处理任务往往需要检查文件系统的某些部分，以了解全局应用状态。相比之下，集群 X 中的应用受用户更明确的控制，通常会预先知道所有所需文件的名称。

7. 经验

在构建和部署 GFS 的过程中，我们遇到过各种问题，其中一些属于运维问题，另一些属于技术问题。

起初，GFS 被设想为生产系统的后端文件系统。随着时间推移，其使用范围扩展到研究与开发任务。它最初几乎不支持权限、配额等功能，但现在已经具备这些功能的初步形式。生产系统通常纪律严明、管理受控，但用户有时并非如此。我们需要更多基础设施来防止用户彼此干扰。

我们遇到的一些最大问题与磁盘和 Linux 有关。许多磁盘向 Linux 驱动程序声明支持一系列 IDE 协议版本，

但实际上只有在较新的版本下才能可靠响应。由于这些协议版本非常相似，这些硬盘大多数时候可以正常工作；但偶尔，版本不匹配会导致硬盘与内核对硬盘状态产生分歧。内核中的问题会因此悄无声息地损坏数据。这个问题促使我们使用校验和来检测数据损坏；与此同时，我们修改了内核，以处理这些协议不匹配的情况。

此前，由于 **fsync()** 的开销，我们在 Linux 2.2 内核上遇到过一些问题。其开销与文件大小成正比，而不是与被修改部分的大小成正比。对于我们的大型操作日志而言，这尤其是在实现 **checkpoint** 之前是一个问题。我们一度通过使用同步写入来规避它，最终迁移到了 Linux 2.4。

另一个 Linux 问题是单个读写锁：地址空间中的任何线程从磁盘换入页面时需要持有它 (读锁)，或者在 **mmap()** 调用中修改地址空间时需要持有它 (写锁)。我们曾在轻负载下观察到系统出现瞬态超时，并仔细排查资源瓶颈或偶发的硬件故障。最终，我们发现，当磁盘线程正在换入先前已映射的数据时，这把单独的锁会阻塞主网络线程将新数据映射到内存中。由于我们的主要限制来自网络接口，而非内存拷贝带宽，我们以额外一次拷贝为代价，将 **mmap()** 替换为 **pread()**，从而规避了这个问题。

尽管偶尔会遇到问题，Linux 代码的可用性仍多次帮助我们探索并理解系统行为。在适当情况下，我们会改进内核，并将这些改动分享给开源社区。

8. 相关工作

与其他大型分布式文件系统 (例如 AFS[5]) 一样，GFS 提供位置无关的命名空间，使数据能够为了负载均衡或容错而被透明地迁移。与 AFS 不同，为了提供聚合性能并提升容错能力，GFS 以更接近 xFS[1] 和 Swift[3] 的方式，将一个文件的数据分布到多个存储服务器上。

由于磁盘相对廉价，并且复制比更复杂的 RAID[9] 方案更简单，GFS 目前只使用复制来提供冗余。因此，它比 xFS 或 Swift 消耗更多原始存储空间。

与 AFS、xFS、Frangipani[12] 和 Intermezzo[6] 等系统不同，GFS 不在文件系统接口之下提供任何缓存。我们的目标工作负载在单次应用运行中几乎没有数据复用，因为它们要么流式处理大型数据集，要么在其中随机寻道，并且每次只读取少量数据。

一些分布式文件系统，例如 Frangipani、xFS、明尼苏达大学的 GFS[11] 和 GPFS[10]，移除了中心化服务器，并依赖分布式算法实现一致性与管理。我们选择中心化方案，以简化设计、提高可靠性并获得灵活性。特别是，

中心化 master 使实现复杂的 chunk 放置与复制策略容易得多, 因为 master 已经拥有大部分相关信息, 并控制这些信息如何变化。我们通过保持 master 状态较小, 并将其完整复制到其他机器上来实现容错。目前, 可扩展性与读取高可用性由我们的 shadow master 机制提供。对 master 状态的更新通过追加到预写日志而持久化。因此, 我们可以采用类似 Harp[7] 的 primary-copy 方案, 提供比当前方案具有更强一致性保证的高可用性。

在向大量客户端提供聚合性能方面, 我们处理的问题与 Lustre[8] 类似。不过, 我们专注于应用需求, 而非构建符合 POSIX 的文件系统, 因此显著简化了这个问题。此外, GFS 假设系统由大量不可靠组件构成, 因此容错是设计的核心。

GFS 与 NASD 架构 [4] 最为相似。NASD 架构以网络附加磁盘驱动器为基础, 而 GFS 如 NASD 原型一样, 使用普通机器作为 chunkserver。与 NASD 工作不同, 我们的 chunkserver 使用延迟分配的固定大小 chunk, 而非可变长度对象。此外, GFS 实现了负载再平衡、复制与恢复等生产环境所需的功能。

与明尼苏达大学的 GFS 和 NASD 不同, 我们并不试图改变存储设备模型。我们关注利用现有普通组件, 满足复杂分布式系统日常的数据处理需求。

原子 **record append** 支持的生产者-消费者队列, 解决了与 River[2] 中分布式队列类似的问题。River 使用分布在多台机器上的内存队列和精细的数据流控制, 而 GFS 使用可由多个生产者并发追加的持久化文件。River 模型支持 m 对 n 的分布式队列, 但缺少持久化存储带来的容错能力; GFS 则只能高效支持 m 对 1 的队列。多个消费者可以读取同一文件, 但必须协调对输入负载的划分。

9. 结论

Google File System 展现了在普通硬件上支撑大规模数据处理工作负载所必需的特性。尽管某些设计决策是针对我们独特环境作出的, 但其中许多决策也适用于规模和成本敏感度相近的数据处理任务。

我们从当前及预期的应用工作负载和技术环境出发, 重新审视了传统文件系统的假设。这些观察使我们在设计空间中选择了截然不同的方案。我们将组件故障视为常态而非例外, 针对主要以追加方式写入 (可能是并发追加)、随后读取 (通常为顺序读取) 的巨大文件进行优化, 并通过扩展和放宽标准文件系统接口来改进整个系统。

我们的系统通过持续监控、复制关键数据以及快速自动恢复来提供容错能力。chunk 复制使我们能够容忍 chunkserver 故障。这些故障的频率促使我们设计了一种新的在线修复机制, 它会定期且透明地修复损坏, 并尽快补偿丢失的 replica。此外, 我们使用校验和检测磁盘或 IDE 子系统层面的数据损坏; 考虑到系统中的磁盘数量, 这类问题已经变得十分常见。

我们的设计能够向执行各种任务的大量并发读取者和写入者提供高聚合吞吐量。我们通过将经由 master 的文件系统控制, 与直接在 chunkserver 和客户端之间进行的数据传输分离, 实现这一目标。大 chunk 大小和 chunk lease 减少了 master 对常见操作的参与; 其中 chunk lease 将数据 mutation 中的权限委托给 primary replica。这使得我们能够采用简单的中心化 master, 同时避免其成为瓶颈。我们相信, 对网络栈的改进将消除当前单个客户端写入吞吐量的限制。

GFS 已成功满足我们的存储需求, 并在 Google 内部广泛用作研究与开发以及生产数据处理的存储平台。它是一项重要工具, 使我们能够持续创新, 并解决整个 Web 规模的问题。

致谢

我们感谢以下人员对系统或论文作出的贡献。Brian Bershad(我们的论文指导人) 以及匿名评审者提出了宝贵意见和建议。Anurag Acharya、Jeff Dean 和 David desJardins 参与了早期设计。Fay Chang 完成了跨 chunkserver replica 比较相关的工作。Guy Edjlali 负责存储配额。Markus Gutschke 完成了测试框架和安全性增强相关工作。David Kramer 负责性能改进。Fay Chang、Urs Hoelzle、Max Ibel、Sharon Perl、Rob Pike 和 Debby Wallach 对论文早期草稿提出了意见。Google 的许多同事勇敢地将数据托付给这个新文件系统, 并提供了有用反馈。Yoshka 参与了早期测试。

参考文献

- [1] Thomas Anderson, Michael Dahlin, Jeanna Neefe, David Patterson, Drew Roselli, and Randolph Wang. Serverless network file systems. In *Proceedings of the 15th ACM Symposium on Operating System Principles*, pages 109–126, Copper Mountain Resort, Colorado, December 1995.
- [2] Remzi H. Arpaci-Dusseau, Eric Anderson, Noah Treuhaft, David E. Culler, Joseph M. Hellerstein, David Patterson, and Kathy Yelick. Cluster I/O with river: Making the fast case common. In *Proceedings of the Sixth Workshop on Input/Output in Parallel and Distributed Systems*, IOPADS '99, pages 10–22, Atlanta, Georgia, May 1999.
- [3] Luis-Felipe Cabrera and Darrell D. E. Long. Swift: Using distributed disk striping to provide high I/O data rates. *Computer*

Systems, 4(4):405–436, 1991.

- [4] Garth A. Gibson, David F. Nagle, Khalil Amiri, Jeff Butler, Fay W. Chang, Howard Gobioff, Charles Hardin, Erik Riedel, David Rochberg, and Jim Zelenka. A cost-effective, high-bandwidth storage architecture. In *Proceedings of the Eighth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 92–103, San Jose, California, October 1998.
- [5] John Howard, Michael Kazar, Sherri Menees, David Nichols, Mahadev Satyanarayanan, Robert Sidebotham, and Michael West. Scale and performance in a distributed file system. *ACM Transactions on Computer Systems*, 6(1):51–81, February 1988.
- [6] Intermezzo. <http://www.inter-mezzo.org>, 2003.
- [7] Barbara Liskov, Sanjay Ghemawat, Robert Gruber, Paul Johnson, Liuba Shrira, and Michael Williams. Replication in the harp file system. In *Proceedings of the 13th Symposium on Operating System Principles*, pages 226–238, Pacific Grove, California, October 1991.
- [8] Lustre. <http://www.lustre.org>, 2003.
- [9] David A. Patterson, Garth A. Gibson, and Randy H. Katz. A case for redundant arrays of inexpensive disks (RAID). In *Proceedings of the 1988 ACM SIGMOD International Conference on Management of Data*, pages 109–116, Chicago, Illinois, September 1988.
- [10] Frank Schmuck and Roger Haskin. GPFS: A shared-disk file system for large computing clusters. In *Proceedings of the First USENIX Conference on File and Storage Technologies*, pages 231–244, Monterey, California, January 2002.
- [11] Steven R. Soltis, Thomas M. Ruwart, and Matthew T. O’Keefe. The global file system. In *Proceedings of the Fifth NASA Goddard Space Flight Center Conference on Mass Storage Systems and Technologies*, College Park, Maryland, September 1996.
- [12] Chandramohan A. Thekkath, Timothy Mann, and Edward K. Lee. Frangipani: A scalable distributed file system. In *Proceedings of the 16th ACM Symposium on Operating System Principles*, pages 224–237, Saint-Malo, France, October 1997.